

CHANGELOG

Changelog

All notable changes to the Claude multi-account toolkit.

v1.26.1 — 2026-07-25 — ccr buildable on same-minor toolchains + launch locks that actually lock + honest toolchain-skew guidance

Patch release on top of v1.26.0. Root-caused from a live install failure (ccr: no router found on a host WITH Go installed) and the launch-path investigation it triggered; every fix carries a RED->GREEN regression test observed to fail on the pre-fix artifact.

Fixed

- **Cross-alias gateway port-collision chain (live-proven: xiaomi and opencode2 both hashed to 3519).** Three links closed: (1) the 100-slot per-alias port spaces collided under ~50 router aliases — widened to 500 slots each (gateway [3460,3959], management [3960,4459]; a surviving collision now fails loudly with rc=78 instead of silently degrading); (2) ccr's waitForReady accepted a FOREIGN gateway's /health as the child's readiness, so a colliding child that died on bind was recorded "ready" in the pidfile — a 200 now counts only when the port's holder IS the child (cmd/ccr/waitforready_test.go); (3) ccr default-claude-code's fresh-start path rebuilt the dead service with environment DEFAULTS, silently reverting the per-alias ports to the shared 3456/3458 — a dead pidfile's recorded flags are now replayed. The claude-sonnet-5 404s seen on opencode2/openrouter were transient fallout of this landscape (default-port gateways serving mismatched configs mid-sweep); with the chain closed those aliases route correctly again.
- **Live-proof classifier learned two more honest classes (it was pinning the suite red on non-toolkit conditions).** (1)

context-inadequate now matches the no-“about” phrasing — live on nvidia3/mistral-small: maximum context length is 32768 tokens. However, you requested 113865 tokens (...) — the branch regex used to REQUIRE “about”, so this genuine provider-side overflow fell through to a counted api-error; extraction is also pinned against the trailing-parenthesis trap (the request is 113865, never the 65536 completion budget). (2) New environment-incompatible KNOWN-NON-WORKING class: the provider’s own 400 rejects an MCP tool NAME from the host’s plugin environment (live on nvidia4: Function name was mcp__plugin_aws-startup-advisor_... but must be ... maximum length of 64) — the toolkit neither mints nor rewrites MCP tool names, so a strict provider’s grammar rejection is not a routing defect; the marker stays on the evidence’s face and a name-sanitizing transform is the tracked follow-up. Both classes: evidence-based, re-derived every run, PASS never reaches them, generic 400s stay counted (covered by new cases in test_layer4_route_attribution.sh, 214/214 incl. a re-scoped mutation anchor).

- **Every router alias shared ccr’s default management port 3458 — only ONE per-alias gateway could be up at a time (live defect, sarvam rc=78).** Per-alias CCR_HOME isolation gave each alias its own gateway port but left the management port at 3458 for all of them, so every second alias’s ccr restart child died with start management interface: listen on 127.0.0.1:3458: bind: address already in use and the launch refused with rc=78 (“route was written but never applied” — live-captured: 4 child starts, 4 bind failures). Worse, the child binds its gateway socket BEFORE the management bind fails, so waitForReady could WIN the race against the dying child — a launch that “succeeded” against a gateway that was already exiting. The launcher now derives a per-alias management port from the same provider-id hash in the disjoint [3960,4459] space (CMA_CCR_MGMT_PORT in the provider env pins it) and passes it to every ccr restart call; ccr default-claude-code’s recovery path replays the pidfile flags, so the port survives restarts. Covered by new test_mgmt_port_isolation.sh (both port spaces, per-alias distinctness, the pin; green on the fix, red on the prefix body).
- **Live route-attribution verifier read the STALE GLOBAL ccr dir; every router proof leg failed attribution (29 FAILS).** verify_superpowers_tui.sh read ~/.claude-code-router/config.json (+ service.json/service.log for the restart receipt) — a layout nothing writes any more since per-alias CCR_HOME isolation landed (5f9d82f, 2026-07-24): router launches write ~/.claude-code-router/<provider-id>/config.json and run their gateway with CCR_HOME pointed at the same per-alias dir. The global file still held a Jul-20 route (deepseek), so every router leg resolved the wrong backend (intended=xiaomi ... resolved=deepseek/deepseek-v4-pro). The verifier now reads the same per-alias dir the launcher writes (gate semantics unchanged: intended-vs-resolved on default+background,

restart receipt fail-closed — only WHERE the files are read).
claude-release-gate.sh had the same drift in its sink-side route
proof and is fixed identically; test_layer4_route_attribution.sh
fixtures moved to the per-alias layout with a stale-global decoy
planted so the drift cannot silently return (205/205 green; the
same suite fails 22 ways against the pre-fix verifier).

- **_cma_ccr_unfossilise leaked the cfg lock fd and the held flock on its early returns.** The CAS-1 mismatch and mktemp-failure paths returned between _cma_cfg_lock and _cma_cfg_unlock; in the operator's INTERACTIVE shell (which never dies) the kernel never releases a leaked flock, so each occurrence held the lock for the shell's life, stacked +1 leaked fd, and stalled every later cfg-lock on that config for the full 5s budget while silently degrading it to unlocked — precisely under the concurrent-launch conditions the lock exists for (found by independent audit, live-proven). Both returns now pass through the unlock; test_lock_fd_persistence.sh covers both paths (fd var cleared, contender acquires immediately, zero fd delta).
- **Bundled ccr no longer refuses to build on a same-minor Go toolchain (install blocker).** The submodule's go.mod pinned go 1.26.4; on a host running go1.26.2 with GOTOOCHAIN=local (distro default on this toolchain build), go build refuses with go.mod requires go >= 1.26.4, so install.sh correctly hard-failed — but the fix text said “install Go” on a host that HAD Go. Patch releases never add language features, so the directive is now go 1.26 (matching the sibling cma-proxy module, which built fine the whole time). Guarded by a new test_ccr_build.sh invariant asserting the directive stays major.minor-only.
- **The eph/cfg launch locks silently never locked — every config/marker critical section ran without mutual exclusion.** _cma_eph_lock / _cma_cfg_lock opened their lock fd as { exec 9>>FILE; } 2>/dev/null: hardcoded LOW fds (9/10) collide with the shell's OWN redirection bookkeeping (bash saves/restores fds around compound-command redirections and command substitutions onto low free fds), so the fd the inner exec opened did not survive the group — proven live on bash 5.2.37: the next flock -w 5 9 failed “Bad file descriptor” (hidden by the group's own 2>/dev/null), the “5s timeout” handler fired spuriously, and the interactive launch leaked bash: line 1: 10: Bad file descriptor onto the operator's terminal. Both locks now use dynamic fd allocation (exec {var} >>FILE — the shell picks a guaranteed-free fd), which also removes the need for the stderr suppression that caused the original permanent-stderr-silencing defect. New scripts/tests/test_lock_fd_persistence.sh extracts the SHIPPED function bodies and proves: lock really held against an external contender, really released on unlock, stderr alive, no bad-fd noise, and two racing writers serialize to the exact count (fails 10 ways on the pre-fix code, including the lost-update race ending at 40 of 80).

- **Misleading “install Go” remediation texts.** `install.sh`’s failure record, `claude-install-verify.sh`’s absent-router failure, and the standalone `ccr-install.sh` now diagnose toolchain VERSION SKEW accurately: they name both remedies (`GOTOOLCHAIN=auto` `claude-ccr-build` when an older Go is present with network access, or upgrade Go) instead of telling operators to install what is already installed. `ccr-install.sh` gained the same up-front skew advisory `claude-ccr-build.sh` already had.

Fixed (ccr submodule, vasic-digital/claude-code-router)

- **test/livereload drift after ATM-870 (graceful route swap).** `a6a24e0` added `gw.SwapConfig` (atomic in-place config swap; /health, /ready and routing all follow it per-request) but the live reload tests still waited for the retired sentinel config reloaded and validated: — all three `TestConfigHotReloadLive` subtests timed out at 30s each. `ValidReload_AcceptLineNamesNewConfig` and `CleanShutdown_WatcherStops` re-sentinelled to the real accept line (config reloaded and applied in-place:); the dead-premise `HonestBoundary_NotSwappedLive` was rewritten as `InPlaceSwap_HealthReflectsNewConfig` (asserts the swapped config IS served live; documents the two things that do NOT follow a swap: the startup-built outbound proxy client and in-flight request snapshots). Stale “only a process bounce applies config” claims corrected in `serve.go`, `reload.go`, `service.go`, `restart_test.go` and the submodule docs (`ADMIN_MANUAL`, `DOC-AUDIT`, `LIVE_TESTING`). `ccr restart` remains the toolkit’s deterministic apply-point; `go test ./...` fully green.

v1.26.0 — 2026-07-24 — free-tier-first per-model aliases by default + dynamic account dispatch + provider-scoped gateway paths

Minor release on top of v1.25.5 covering the ATM-850/851/852/853/860 defect batch (2026-07-23/24). Every fix carries RED-on-the-broken-artifact -> GREEN -> a paired \$1.1 mutation observed to FAIL, plus live captured evidence where a live path exists (`qa-results/toolkit_defects_20260723/`).

Added

- **Free-tier-first per-model provider aliases are now the DEFAULT sync behaviour (ATM-860, operator decision D14).** The per-model verification + multi-alias pipeline

(model_verify.py + providers_generate.py) — previously reachable only through the opt-in `sync --multi` that no default path ever invoked (a CONFIGURED-but-not-IN-USE gap, §11.4.196(F)) — now runs as part of every plain `claude-providers sync`, restricted to FREE-tier models. Free-vs-paid derives from REAL data only, never a roster: `models.dev` cost rows (0/0 on both sides = free), the provider `:free` id convention, and — for models no commercial catalog knows — a self-hosted loopback/RFC1918 endpoint. A model whose tier is underivable is treated as PAID and skipped unprobed (fail-safe on spend); skips are recorded honestly in `<provider>_verified.json` (free_only, skipped_models with per-model tier + reason), never as failures. Paid probing is an explicit opt-in: `sync --include-paid` or `CMA_SYNC_INCLUDE_PAID=1`; `CMA_SYNC_MULTI=0` restores the legacy single-alias-only default. Verification remains a REAL completion round-trip per model (existence + VERIFY_OK sentinel + tool-call + streaming probes) — a models-list entry is not evidence, and a listed-free model whose completion fails is never admitted. Alias generation is idempotent + re-runnable with deterministic, collision-free naming (provider, provider2, ...): ranking and pairing now tie-break on model_id, so equal-score models can no longer swap aliases between runs on concurrent-probe completion order (verification cache bumped to v3). Native-first ordering (§11.4.196(A)) is structural and unchanged: generated aliases are provider-class (`cma_run_provider`, `~/.claude-prov-*`), a namespace the account-class dispatcher refuses. Live proof: default sync against the real opencode free tier (`models.dev` fetched live; free models probed with real completions, paid/unknown skipped unprobed, second run byte-identical).

- **Bare account names dispatch dynamically — claude5 works in shells older than its alias file (ATM-850).** A `command_not_found_handle(r)` (bash+zsh) emitted into the managed alias block resolves `<name> -> ~/.claude-<name>` at INVOCATION time, so a tmux pane shell that sourced an older `aliases.sh` still launches a newly-added account out of the box. Defined names always win; a pre-existing handler is chained; `prov-*/code-router` are refused; a non-account miss keeps `rc=127`. `claude-add-account` now detects live tmux servers holding stale pane shells and prints the exact re-source/broadcast commands (broadcast only on an interactive default-No confirm — never autonomous send-keys).

Fixed

- **Pinned helixagent facade models survive a live sync (ATM-851).** When the live `/v1/models` listing did not contain the pinned facade id, the positional fallback overwrote the explicitly pinned `CMA_HELIXAGENT_STRONG/FAST` with the endpoint-reported `.gguf` path. Pin provenance is now captured before built-in defaults: an explicit pin is authoritative; the live listing

(still fetched as reachability evidence) never overwrites it; unpinned selection stays data-driven.

- **Provider-scoped gateway paths (ATM-852, ccr submodule).** The ccr gateway now serves `/:provider/v1/{messages,chat/completions}` — the provider name is carried in the base URL path, validated against configured providers (unknown segment = 404), and stripped before the protocol classifier; bare `/v1/*` and `/proxy/v1/*` are untouched. The router-transport launcher exports the provider-scoped base URL. Activation on the running gateway requires `claude-ccr-build` + an operator-coordinated ccr restart.
- **Opencode context-compression loop (ATM-853).** `derive_limits()` read only `limit.context` and ignored `limit.input`, so big-pickle's 200k-context/160k-input catalog row produced a compact window ABOVE the model's real input cap — the client-side compact guard could never fire before the endpoint rejected, looping `reject->compact->reject`. `derive_limits` now clamps `ctx = min(limit.context, limit.input)` when the catalog publishes a smaller real input cap; big-pickle resolves (160000, 8192) and the guard fires first. On-alias loop-gone retest remains pending a coordinated ccr window.
- **Uncatalogued live-proven models are no longer demoted on an absent catalog row (ATM-860 finding 2).** `model_verify.py` read an ABSENT catalog context as 0 and failed the `< 8000` gate — a capacity-vs-unknown false refusal (§11.4.201(9)) that zeroed helixagent's whole live-enumerated roster. Only a KNOWN small context demotes; unknown earns no context score but keeps the live-probe verdict.

Notes

- Regression state at draft time: see `qa-results/toolkit_defects_20260723/evidence/d6_regression_sweep.log` (providers suite, helixagent 46, pins 10, dispatch 8, tmux-notice 12, free-tier multi-sync 28, ccr go test `./...`).

v1.25.5 — 2026-07-23 — fail-loud install over broken/missing ccr + interactive-shell stderr fix + anti-fossil r-m-w hardening

Patch release on top of v1.25.4. Every change independently reviewed to a zero-finding / zero-warning GO on Fable-xhigh (Opus-xhigh fallback while Fable is weekly-limited; §11.4.209/§11.4.134); install suite 46/46 green (`suite_rc=0`).

Fixed

- **install.sh no longer prints [done] installed over a broken or missing ccr.** The build/verify seam now probes the ARTIFACT through its real invocation path (not a build-prerequisite's presence): a usable resident ccr with no Go toolchain passes with an honest will-go-stale warning; a broken / npm-doppelganger / wedged / absent ccr hard-fails exit 1 instead of a false success (§11.4.201(11)).
- **The probe watchdog no longer stalls install/verify by +15s on a healthy host.** A watchdog subshell spawned inside a \$(...) command-substitution held the pipe write-end via its orphaned sleep, so every seam blocked to the full timeout budget on instant work. The watchdog's fds are redirected away from the cmd-subst pipe; the fast path returns in ~0s and a genuinely-wedged probe is still bounded at rc=124. (Codified as shell-instrument footgun I7 in the constitution.)
- **Bare exec N>>file 2>/dev/null in the sourced eph-lock helper no longer silences the interactive shell's stderr for the shell's lifetime.** The side redirection is brace-scoped; the eph-lock wait is bounded (best-effort, proceeds unlocked with an honest log on timeout).
- **Anti-fossil marker read-modify-write hardened** against lost updates + word-split.

Verified

- Provider helixagent (local Qwen3-Coder-30B on 127.0.0.1:18434) re-verified + proven end-to-end: model_verify.py anti-bluff scoring PASS + a live captured answer correctly naming the working repo (no --force, no bluff). ## v1.25.4 — 2026-07-22 — un-fossilise ephemeral cma-proxy addresses (kill the 502 class) + toon Go port

Patch + feature release on top of v1.25.3.

Fixed

- **Ephemeral cma-proxy address no longer fossilises into Router.default.** A transform-declaring provider gets a cma-proxy on a scan-until-free port and its base is rewritten to that proxy address — which was then persisted into the durable config.json and left behind when the proxy died on exit, so Router.default named a dead 127.0.0.1:<port> with no listener and every gateway consumer hit 502 dial tcp ... connection refused while the provider's real backend was healthy. Two compare-and-swap guards now (1) repair our own ephemeral address back to the real endpoint on exit and (2) reap fossils whose owning launch died — reaping only a provably-dead holder (kill -0, §11.4.180) and never clobbering a concurrent

live route; the real endpoint is kept in a sidecar marker, not config.json. Measured: helixagent and poe had both fossilised on 127.0.0.1:3457 and were misread as provider outages. Live re-verification post-fix: all 8 valid aliases launch clean (PING-OK) and poe surfaces its real upstream status instead of a dead-port 502.

Added

- **scripts/toon/ — Go port of toon_encode.py** (added alongside the Python, which remains the sole caller). Byte-parity with the real python3 encoder over the realistic numeric + structural input space (42 golden vectors + a 4000-iteration float-repr sweep), with two honestly-documented, test-gated divergences (bare NaN/Infinity; the findToonScript walk-up superset).

v1.25.3 — 2026-07-22 — trim knob survives generator regen + operator docs for trim/gate/container-mode

Patch follow-up to v1.25.2.

Fixed

- **CMA_PROVIDER_TRIM now survives a provider re-add.** The v1.25.2 trim knob was a hand-added line on the provider .env, which the generator (cma_provider_write_env) would have dropped on the next claude-providers add/sync (the file is machine-generated). The generator now READS the existing CMA_PROVIDER_TRIM value before it truncates the file and re-emits it — matching the existing “preserve-existing-value” pattern; emitted only when set, and the ambient var is unset first so a shell export can never leak onto the file. (scripts/tests/test_provider_trim_persist.sh, red→green.)

Added

- **Operator documentation** for the v1.25.2 features in docs/Provider_Aliases_User_Guide.md (+ synchronized HTML/PDF/DOCX) and docs/Provider_FAQ.md: CMA_PROVIDER_TRIM='bare' trim mode, the claude-release-gate live gate, and the helixagent HelixLLM container-mode dependency (claude mode = one 229k slot vs coder mode = 8×3072).

Changed

- GitLab upstream repointed to the renamed `claude_toolkit` (underscore) repo (upstreams/GitLab.sh + local remotes) — GitLab, like GitHub, has completed the §11.4.29 `snake_case` rename. GitFlic/GitVerse still host the dash name (underscore absent there) and remain on dash until renamed operator-side.

v1.25.2 — 2026-07-22 — ccr resolved by stable path (PATH-shadowing doppelgänger), provider trim mode, mandatory live release gate

Patch. A second field failure on the v1.25.1 shape, with a DIFFERENT root cause the self-heal could never fix: the `npm @musistudio/claude-code-router` package installs its own `ccr` into `nvm's bin dir`, which precedes `~/.local/bin` on `PATH`. Its `--help` carries the same “`ccr start / ccr serve`” fingerprint (it passes the identity gate) but it has NO restart subcommand — so every router launch failed exactly like a stale bundled build, the self-heal rebuilt the BUNDLED binary (which was never the one being invoked), retried bare `ccr restart`, hit the doppelgänger again, and refused. A rebuild cannot fix PATH shadowing. Investigation of the live helixagent chain then surfaced two more stacked defects: the HelixLLM container serving $8 \times 3,072$ -token slots (`-c 24576 --parallel 8 — llama.cpp splits -c across slots`), and $\sim 330k$ tokens of auto-resumed session history + $\sim 110k$ of plugin/MCP tool schemas overflowing the local model's window on every launch (a direct `claude --bare -p hi` request measures 4,891 bytes — the client was never the problem).

Fixed

- **`cma_run_provider` resolves OUR router by its stable install identity, never by PATH order.** Resolution: `$CMA_CCR_BIN` override, else `$HOME/.local/bin/ccr` (the symlink `claude-ccr-build` maintains), `PATH` only as a last resort when the bundled install is absent — and the resolved path is used for EVERY invocation (identity probe, restart, post-rebuild retry, and the launch itself). A PATH-shadowing doppelgänger can no longer brick router aliases nor masquerade as a stale bundled build. (scripts/tests/test_ccr_path_shadowing.sh, executed red→green against the real generated wrapper.)

Added

- **CMA_PROVIDER_TRIM='bare' — per-provider minimal-launch mode** for local-model providers: conversation launches get `--bare` (drops the hook/plugin/MCP/CLAUDE.md surface) and BOTH history seams stay closed — the conversation-args `--resume` auto-injection AND the interactive (zero-args) stored-session-flags injection are skipped, so every launch is a fresh session that actually fits a local window. Explicit user session selectors (`--resume`, `--session-id`, ...) are honored verbatim; non-conversation subcommands are untouched; untrimmed providers are byte-identical to before. Wired for helixagent (whose HelixLLM backend belongs in `helixllm-mode.sh` `claude` — one 229,376-token slot). (`scripts/tests/test_provider_trim.sh`, 11 scenarios red→green.)
- **claude-release-gate — the mandatory LIVE pre-release gate.** Sandbox suite + a live smoke that drives the REAL generated alias through the REAL PATH → `ccr` → `route-apply` → `proxy` → provider backend, asserting the served reply and the sink-side route (`--verify-providers` adds the LLMsVerifier scan). The sandbox suite is structurally blind to real-host state — this release's defect class shipped green through it — so releases now require the live layer. See README "Releasing".

v1.25.1 — 2026-07-22 — router aliases refused to launch on a stale bundled ccr (self-healing rebuild)

Patch. A field failure caught immediately after v1.25.0: from a normal interactive shell, EVERY router-transport provider alias (helixagent, poe, kimi, ...) could refuse to launch —

```
claude-providers: refusing to launch <id> – its ccr route was NOT applied.
```

```
'ccr restart' failed (rc=1): Profile "restart" was not found or is disabled.
```

Fixed

- **A stale bundled ccr binary no longer bricks router-alias launches.** The vendored Go router `ccr` is a gitignored BUILD ARTIFACT, so a submodule bump that added the `ccr restart` subcommand — which every router launch runs to apply its route — does NOT rebuild an existing install's binary. A stale `ccr` parsed `restart` as a profile NAME and replied `Profile "restart" was not found or is disabled (rc=1)`; the launch's fail-safe then (correctly, so it never serves the wrong model) REFUSED — but that refusal bricked every router alias at once, with an opaque message. `cma_run_provider` now SELF-

HEALS on exactly that shape: it rebuilds once via `claude-ccr-build` and retries `ccr restart`; only if that cannot resolve it does it refuse, now with an actionable “rebuild it: `claude-ccr-build`” message. The heal is bounded (one rebuild, one retry), gated on the stale-binary shape alone (a 401/402/403 auth failure or a timeout is NOT a rebuild trigger and still counts), and fail-closed if the rebuild does not help. Regression-guarded by `test_ccr_restart_selfheal.sh`, which EXECUTES the real wrapper against a stale-ccr stub (not a text grep): self-heal fires + retries, the launch proceeds after a successful heal, an actionable fail-closed refusal when it cannot, and non-stale shapes excluded.

Verification gap that let it ship (honest)

- v1.25.0’s live verification launched helixagent through the scrubbed-env `verify --deep / superpowers-TUI` path AND ran the run-proof AFTER the ccr binary happened to be rebuilt — so it never exercised a plain interactive `<alias>` launch against a *stale* build artifact, which is the exact path that failed for the operator. Green internal-harness tests masked a broken user-facing path. The regression test above now covers that path.

Verified

- **Hermetic sandbox suite: 42 files, 42 passed, 0 failed — ALL GREEN.** Includes the new `test_ccr_restart_selfheal.sh` (9/0), which EXECUTES the real `cma_run_provider` against a stale-ccr stub and pins: the self-heal fires + retries, the launch proceeds after a successful heal, an actionable fail-closed refusal when it cannot, and — a real behavioral Scenario C, not a literal grep — that a NON-stale restart failure (a `CCR_API_KEYS` auth refusal) does NOT trigger a rebuild and is not mislabeled. `test_ccr_conformance.sh` 12/0; `test_coverage.sh` 70/0 (its SIGPIPE-pipeline lint first caught the new test).
- **The live install:** ccr rebuilt from the released submodule (904effb); `ccr restart` → `rc=0`; a direct interactive helixagent launch now passes the ccr-restart gate (reproduced) instead of refusing.
- **Independently reviewed SOUND:** the self-heal is trigger-specific (stale shape only; auth/5xx/timeout still count), bounded (one rebuild, one retry), and fail-closed (a failed rebuild returns 78 before `ccr default-claude-code`, never serving an unapplied route).

v1.25.0 — 2026-07-22 — the compatibility proxy goes Go (cma-proxy) + HelixAgent fully working (routed + capacity-sized + tool-engaging) + provider-verification honesty + rc-safety

Minor release. Two headline changes make it a minor:

1. **A new user-facing component:** the toolkit's provider-compatibility proxy is now a single Go binary, **cma-proxy** (scripts/proxy/, module cmaproxy), consolidating the three former per-provider python proxies (poe/kimi/sarvam) plus a net-new helixagent tool-call transform. Python is fully removed from the proxy layer.
2. **HelixAgent goes from a broken facade to a working local provider** — genuinely routed (the :18434 repoint), capacity-sized (the 229376 pin), and **tool-engaging** (the Go proxy's Hermes tool-call recovery). A live, route- attributed verify helixagent --deep turn in claude mode PASSED end-to-end.

Alongside those it carries provider-verification honesty (a context-inadequate classification so a too-small backend is not miscounted as toolkit breakage), a shell-rc safety fix (§11.4.167 — the toolkit can no longer erode a user's .bashrc), and repo/CI hygiene. The GPU mode-switch that reallocates the shared HelixLLM backend between HelixCode and Claude Code still ships in the companion helix_code repo, not this toolkit. **Version chosen: v1.25.0** (a new shipped component plus a previously-dead provider made to work is additive, backward- compatible functionality — see the closing note).

Proxy framework — python proxies replaced by one Go binary (cma-proxy)

- **The three former python proxies (poe_proxy.py, kimi_proxy.py, sarvam_proxy.py) are gone, folded into ONE Go binary cma-proxy** (scripts/proxy/, module cmaproxy) — which ALSO carries a net-new helixagent transform. (There was never a helixagent_proxy.py in the repo: helixagent's prior break was routing/base_url (the :3456→:18434 repoint), not tool-format; its Hermes recovery is brand-new, written directly in Go.) Motivated by an operator directive that the toolkit carry no python; a streaming HTTP proxy is not bash-appropriate, so Go — which also matches the already-Go bundled ccr.

- **Structure** (each provider is a self-contained file that registers itself in an `init()`, so a new provider needs no edit to `main.go`):
 - `hermes.go` — helixagent Hermes/Qwen tool-call recovery (response-side; `registerResponse("helixagent")`).
 - `poe.go` / `kimi.go` / `sarvam.go` — request-schema fixes (`registerRequest`): Poe tool-param injection + `$ref/$defs` resolve + ~216 tool-count cap + `cache_control` strip; Kimi moonshot-flavored `#!/$defs/` schema normalization; Sarvam content-block flatten + `max_tokens` tier clamp.
 - `registry.go` — `init()`-time registration plus family resolution (providerKey: exact id, then id-up-to-first-digit `poe2` → `poe`, then id-up-to-first-`kimi-for-coding` → `kimi`).
 - `main.go` — HTTP server: request-transform-then-forward → response-transform (only on a 200, only for a response-transform provider) → otherwise verbatim passthrough; a clean **502-JSON** body on any upstream/connection error (never an empty reply); and the `--has-transform <id>` discovery gate (exit 0/1).
- **Correctness win over the python it replaces:** the Hermes parser is now **delimiter-robust** — blocks split on the OPENING tags (`<function=`, `<parameter=`), a parameter value ends at the *last* `</parameter>` in its segment, and a balance guard bails to passthrough on unbalanced opening tags — so a tool-arg VALUE that itself contains `</function>` or `</parameter>` (e.g. Write-ing a file *about* tool-calling) is preserved verbatim instead of being silently truncated/dropped, which is exactly what the python parser did. Found in review (2026-07-22) and pinned by a Go regression test.
- **Wiring:** `claude-proxy-build.sh` builds + installs `cma-proxy` (mirroring `claude-ccr-build.sh`: `go build` → copy into `$SHARED_DIR/proxy` → symlink onto `PATH`, with a `--has-transform` self-check); `install.sh` §4b now runs that build (was: copy `scripts/proxy/*.py` into the shared store); and `cma_run_provider` gates on `cma-proxy --has-transform <id>` then launches `cma-proxy --provider <id> --port <port>` with the upstream (`CMA_PROVIDER_BASE_URL`) exported inline to the child. `install.sh` already treats the proxy build as best-effort (Go-gated, like `ccr`): with no go, install still completes and the proxied aliases fall back to their direct endpoint with the compat shims INACTIVE.
- **Tests:** co-located Go tests (`poe_test.go` 11, `kimi_test.go` 12, `sarvam_test.go` 8, plus `hermes_test.go`'s recovery/passthrough/`</function>`-in-value regression cases) driven by the new `scripts/tests/test_cma_proxy.sh` (`go build` + `go vet` + `go test` + `gofmt` + the `--has-transform` family gate). The python-only proxy tests were removed (`test_poe_proxy.sh`, `test_sarvam_proxy.sh`) and `test_kimi.sh`'s proxy section migrated.
- **Honest scope:** this migrates ONLY the proxy layer. The remaining python — six tooling scripts (`providers_resolve.py`, `model_verify.py`, `opencode_sync.py`, `providers_generate.py`, `toon_encode.py`, `alias_e2e_test.py`) plus two test-lib helpers

(tests/lib/classify_live.py, tests/lib/pty_drive.py) — is a planned follow-up, **not** in this release.

Fixed

- **HelixAgent was a non-attributable facade — now genuinely routes to the real HelixLLM server, and a mode-switch makes its big-context turn actually serveable.** scripts/providers/helixagent.json pinned base_url to the ccr gateway itself (127.0.0.1:3456) — self-defeating, since a router provider cannot route through the gateway it is: cma_run_provider's self-reference guard refuses it and v1.24.0's route-attribution gate correctly marks it failed (before those gates it was badged verified on turns actually served by whichever provider ran last — the exact bluff v1.24.0 exposed), so after the v1.24.0 sync it dropped out of claude-providers list. Repointed at the real backing server 127.0.0.1:18434 (the OpenAI-style endpoint the operator's local HelixLLM / Qwen3-Coder-30B actually serves): confirmed live (a real chat call returns 200 with the existing key), Gate 0 passes (base $\neq$ gateway), and claude-providers verify helixagent returns verified at the probe layer. The remaining obstacle was capacity, not routing: one 32 GB GPU cannot simultaneously serve HelixCode's eight concurrent 3072-token slots and Claude Code's large single-slot request. That is resolved by a **mode-switch** (helix_code/scripts/helixllm-mode.sh) that flips the shared HelixLLM container between two mutually-exclusive modes on the one GPU, one at a time: **codier** (-c 24576 --parallel 8 — eight 3072-token slots, serving HelixCode) and **claude** (-c 229376 --parallel 1 — one large slot, serving the toolkit helixagent alias / Claude Code). The helixagent pin's context_limit is corrected to **229376** (was 180224 in the prior draft, itself a correction of the old per-slot value): 180224 fit Claude Code's first ~67K request, but once tools actually fire (see the proxy bullet below) the multi-turn agent loop accumulates to ~182,128 tokens, which overflowed a 180224 slot with the exact 400 ... exceeds the available context size (180224 tokens) on ~2 of 3 runs. 229376 gives headroom — the auto-compact window still operates at its 200000 design cap with ~21K slack — and it fits VRAM, measured live in claude mode at 30,244 MiB used / 1,854 free / 32,607 total (the KV estimate VRAM $\approx 18128 + 0.053 \cdot \text{ctx}$ MiB validated across 24576→19434, 180224→27676, 229376→30244). A live, route-attributed verify helixagent --deep turn in claude mode proves the two routing/capacity things the pin exists to fix: the alias genuinely reaches HelixLLM (ccr's resolved route matches intent for BOTH the foreground and the background request, with a restart receipt bracketing the launch — not another backend), and Claude Code's real system+tool request (66,693 input tokens) fits the 229376 slot with zero context overflow — the old per-slot 400

exceeds context that demoted it is gone. Because HelixCode is the common case, the **default release state is coder mode**, so helixagent ships demoted to unverified (shown in list-faulty, refused by the launch gate) until the operator switches HelixLLM to claude mode (`helixllm-mode.sh claude`) — this is provider-side capacity to be reallocated, not toolkit breakage, and the suite classifies the coder-mode deep-turn overflow as a distinct context-inadequate class (see Reliability), never an account or routing failure. An audit confirmed helixagent was the ONLY provider with this facade misconfiguration and that a plain sync cannot re-break the repoint (the pin is the single source; the gates fail closed). (f72a756; two stale “base_url → :3456” comments corrected in 9bb6f1f, all other :3456/facade references left as correct history of the incident that motivated the guard.)

- **The tool-call format gap is fixed by the new Go proxy (cma-proxy, scripts/proxy/hermes.go), so Claude Code’s tools actually fire in the HelixAgent path.** Root-caused live: the container already runs llama.cpp with `--jinja`, and a direct request to :18434 returns proper structured tool_calls. But Claude Code’s system prompt induces the model to write a conversational preamble BEFORE the call, in Qwen’s native Hermes/XML form (`<function=NAME><parameter=P>V</parameter></function></tool_call>`); llama.cpp’s parser only extracts a call that *leads* the generation, so once prose precedes it the whole thing is returned as content text (`finish_reason:"stop", tool_calls:null`) and Claude Code never engages. cma-proxy sits inline (`ccr → proxy → :18434`) — the launch wrapper starts it when `--has-transform helixagent` matches — buffers the response, and ONLY when it finds a complete Hermes block llama.cpp did not already parse rewrites it into structured tool_calls (streaming and non-streaming), keeps the preamble as content, coerces each parameter by the request’s own tool schema (a string stays a string; integer/number/boolean/object/array parsed), and passes everything else through byte-for-byte untouched so the common path is never altered. Proven hermetically with no network or GPU: `hermes_test.go` (streaming
 - non-streaming recovery, passthrough safety, and the `</function></parameter>` in-value regression) plus `test_cma_proxy.sh`’s build/vet/gofmt/go-test gate all pass, and a live unit request through the proxy turns the exact leaking generation into a clean `Read({"file_path":"README.md"})`. End-to-end, the strict layer-4 engagement gate now records a **verified** result: a live, route- attributed verify helixagent --deep turn — launched through the real wrapper, which started the Go cma-proxy — PASSED, the model returned the exact skill-content challenge answer (“Skills evolve. Read current version.”), the route resolved to helixagent for both the foreground and the background request, input grew to 143,776 tokens and fit the 229376 slot, and status flipped

to verified. Reliability at 229376 is **2 of 3** (the third miss was a slow-turn timeout of the local 30B, not an overflow — the 180224 overflow is gone). Because coder is the default operational state, helixagent still ships demoted to unverified until the operator switches HelixLLM to claude mode. `claude-proxy-build.sh` builds `cma-proxy` and `install.sh` §4b runs it, so the proxy ships on the normal release path.

Reliability

- **run-proof honestly classifies a context-inadequate backend instead of counting it as fresh toolkit breakage.** A verified router provider whose backing model returns a context-overflow 400 on the large, tool/skill-heavy layer-4 turn is now its own KNOWN-NON-WORKING class — beside account-dead and route-integrity — reported # FAIL: context-inadequate (backend M tokens < request N) and NOT counted as a suite failure: it is provider-side (relaunch a local backend larger, or pin a larger-context model for a hosted one) exactly as an unfunded key is (top it up). **Two** live overflow phrasings are recognized — the llama.cpp shape (local backends, e.g. a HelixLLM in coder mode, whose per-slot context is small) and the OpenAI/OpenRouter shape (hosted models, e.g. an OpenRouter model whose window is under Claude Code’s request) — with phrasing-aware extraction, since the two put the request/window numbers in *reversed* order and the marker must never swap them. This is a state of a backend, not a permanent property of any provider — a mode-switch or a larger pin removes it. The counts are read from the live 400, never a pinned/declared context, and the verdict is re-derived from the live layer-4 error on every run (not a persisted status a re-sync could overwrite), so it is durable and cannot false-positive a provider that genuinely answers layer-4. Paired-mutation proven both directions; the existence layer names the same distinct reason. (Follow-up: tune the output-reservation guard so a hosted model whose window is only modestly above Claude Code’s tool-heavy input can *work* rather than be excused.)
- **run-proof no longer counts an account-side 402/403 as toolkit breakage — the billing analogue of the context-inadequate class.** A provider funded when the ~512-token layers-1/2 probe verified it, but whose balance depletes (or key is revoked / access suspended) before the large layer-4 turn, returns a 402 (“Insufficient balance”) or 403 on a *correctly-routed* turn — a fresh red on every proof run for a condition the toolkit cannot fix. A 402/403 can only come from the provider’s billing/authz, never from how a request was formed (a malformed request is a 400, which still counts), so it is now its own KNOWN-NON-WORKING class — # FAIL: account-side (HTTP 402|403 ...), reported on its face (top up or re-key the account) and swept-exempt like context-inadequate. It is

ordered *below* the route-integrity gate (a mis-routed turn still counts) and matches only 402/403 — 401 (a toolkit-attributable bad auth header), 429, 500, and timeouts still fail the suite. Evidence-based (the live status, never a persisted one), and it cannot false-positive a paying account. Found live on this release's own proof: `inference/glm-5.2` returned "402 Insufficient balance" on a route-attributed turn. (Guards it: `test_providers_gate.sh` — 402 detected, a 400 overflow NOT swallowed, sweep-exempt.)

Safety — shell-rc writes (§11.4.167)

- **The toolkit can no longer erode a user's .bashrc/.zshrc.** A pre-v1.24.0 `prune`→ensure race (fired on every non-interactive shell via `BASH_ENV`) could strip an rc file's body — orphaned managed-headers accumulating while the user's own exports were lost with no backup (this is what erased a set of local env vars during v1.24.0 development). v1.24.0 fixed the main `prune` path; this release closes the **two residual rc-write paths** that still bypassed it: the `install.sh migrate rewrite` (now gated + backup-first, and a silent last-line drop fixed) and the `claude-bootstrap.sh append` (now backup-first). Every rc write now funnels through one committer (`cma_rc_safe_rewrite`) mirroring the alias committer — a no-op guard, a sanity gate that parks a content-losing candidate as `.rejected.<ts>`, a mandatory pristine `.cma-orig` backup-or-refuse, and an INT/TERM-masked publish — so no rc write can proceed without a recovery point. Verified by four paired-mutation tests (rc-safety 35/0) that each fail when a guard is removed. (Follow-up: mint the committer's temp adjacent to the rc so the publish is a true same-filesystem rename.)

Repo & CI hygiene

- **GitHub push-protection no longer trips on the redaction test's fixtures.** `scripts/tests/test_redact.sh`'s 23 synthetic provider-token fixtures were high-entropy random strings, so GitHub flagged two as real credentials and blocked the v1.24.0 push. Every synthetic token is now an obviously-fake `FAKE+zero-padding` body — each token's exact length, prefix, and separator preserved so redaction behaviour and the mutation topology are byte-for-byte unchanged (46/0, both mutations retain teeth) — and the `sk-underscore` fixture uses `sk_not sk_live` (GitHub's Stripe detector matches the `sk_live` prefix on pattern alone, entropy-immune). No pattern-flaggable token remains. (80b07ff, 084a77d)
- **Stray build binaries untracked; LLMsVerifier GEMINI.md added.** Two ~24 MB Go build outputs (fixed-challenge, model-verification) tracked in the LLMsVerifier `llm-verifier/` subdirectory — re-added by an Auto-commit after a prior BFG

purge, referenced by no script — are `git rm --cached + gitignore` (CONST-053); the stray root ccr build binary is gitignored in the router submodule; and LLMsVerifier's missing `GEMINI.md` (§11.4.157 lockstep gap) was added as a faithful Gemini-CLI mirror of its `QWEN.md` (same 135-anchor set through §11.4.167). (3b21f79)

- **CHANGELOG accuracy.** The v1.24.0 “Verified” section cited `test_providers.sh` 294/0; the shipped proof log shows 405/0 (tests were added since the credit-aware work). Corrected to match the on-disk evidence. (8e8fb96)

Submodules

- LLMsVerifier af8d703b → bb729f2b (binary untrack + `GEMINI.md`)
- claude-code-router 6d787fd → 904effb (gitignore stray ccr binary)

Verified

- **Hermetic sandbox suite: 41 files, 41 passed, 0 failed** (`run-all.sh`, ALL GREEN) — including the new `test_cma_proxy.sh` (Go proxy: `go build + go vet`
 - `go test` across `hermes/poe/kimi/sarvam` + the `--has-transform` family gate) and `test_providers_gate.sh` (the `context-inadequate` + new account-side classifiers, 17/0). The migration's three fallout failures — `test_providers`'s stale `_family_id` marker, `test_sandbox_hygiene`'s vacuity flag on the `gofmt` assertion, and the §11.4.157 governance-doc lockstep — were each root-caused and fixed.
- **Go proxy unit tests: `go test ./... in scripts/proxy` → ok** (`hermes` 14, `poe` 11, `kimi` 12, `sarvam` 8); `go vet clean`; `gofmt clean`.
- **HelixAgent live end-to-end, claude mode (229376): layer-4 PASS via the Go cma-proxy.** A route-attributed `verify helixagent --deep`, launched through the real wrapper (which started `cma-proxy` on `:3457` → `:18434`), returned the exact skill-content challenge answer, resolved to `helixagent` for both the foreground and background request, with Claude Code's 143,776-token request fitting the 229376 slot and no overflow — status → verified. VRAM measured live at 30,244 MiB used / 1,854 free of 32,607.
- **Full run-proof.sh — ALL GREEN, exit 0** (coder default): `hermetic` 41/41; live provider-alias verification 21/0; live alias TUI 8 PASS / 0 FAIL / 13 SKIP-GATED (TOTAL 21); alias end-to-end 8/8; `constitution` 7/0. In coder default, `helixagent` is correctly reported `KNOWN-NON-WORKING` `context-inadequate` (backend 3072 < request 67 966), and the new account-side classifier reclassified `inference/glm-5.2`'s live 402 `Insufficient balance` — plus every unfunded/rejected provider — as `KNOWN-NON-WORKING`, so no account-side condition

pins the suite red and no toolkit regression is masked.
Evidence in scripts/tests/proof/ (PROOF.md + per-leg logs).

Version note: **v1.25.0 (minor)** — this release adds a new user-facing component (the Go cma-proxy, replacing the python proxy layer) and, with its Hermes recovery, closes the last gap keeping helixagent from working, taking that provider from a non-attributable broken facade to a routed, capacity-sized, tool-engaging local provider. Semver reserves a minor for backward-compatible added functionality; that is exactly this. It ships.

v1.24.0 — 2026-07-21 — credit-aware model selection; and the v1.23.0 launch regression + the false “ALL GREEN” claim that hid it

Two things ship together here. The **new capability** is credit-aware model selection (below). The **regression** is that v1.23.0 made the bundled Go claude-code-router the sole router but never implemented the launch grammar the toolkit speaks, so **every router-transport provider alias was dead at launch** for the whole life of that release — and it shipped anyway because the verification suite counted a layer-4 FAIL as a pass and no gate read the evidence it had just written. Both the capability and the regression (with the false claim that hid it) are documented below. This is a minor bump because it adds a new user-facing capability, not only a fix.

Added — credit-aware model selection (mandatory tier policy)

- **Every provider alias now picks its model by whether the account can be billed.** Credit / purchased tokens available ⇒ the strongest *paid* model the provider serves that verifies; no credit ⇒ the strongest *free* model; credit state *unknown* ⇒ treated conservatively as no-credit ⇒ the free choice. The unknown branch is deliberately conservative and asymmetric: picking a paid model on an unfunded key fails at launch with 402/403 and leaves a dead alias, whereas picking a free model on a funded key only gives up capability and the next sync corrects it. A human strong_model/fast_model pin (or model_policy=free|paid) in providers/overrides.json still wins; the tier logic only decides when nothing is pinned. Implemented in scripts/providers_resolve.py (tier_preference, model_cost_tier, select_models); the matching credit/cost-aware selection capability landed in the LLMsVerifier submodule

(llm-verifier/{providers,scoring,selection}), kept project-agnostic per its decoupling rules.

- **Free vs paid is classified from real catalog pricing, and “zero-cost” is not assumed to mean free.** A catalog cost: {input:0,output:0} is often a subscription/plan-gated entry that needs a specific key — classifying it free would pick a model that fails at launch. Models with missing or partial pricing are reported as tier unknown, never guessed as free; the OpenRouter :free id suffix is honoured. Deep research established that reliable programmatic balance APIs are rare across providers (most are console-only), so credit state is inferred from the verification probe’s response — 402/403 ⇒ no credit, 429 ⇒ transient (never demotes), 401 ⇒ bad key — with a schema-versioned 24h cache so stale results are never replayed. Covered by scripts/tests/test_provider_credit.sh (127 assertions) and the providers/credit-endpoints.json signal table.
- **Cost note (honest trade-off):** credit detection adds one bounded paid-model probe per provider (30s cap), so the *first* full claude-providers sync after a cache reset is slower than before; the result is cached for 24h, so subsequent syncs skip it. The probes are bounded and never hang; a slow provider is a duration cost, not a stall.

Fixed

- **Token-limit guards were derived independently and could not both be satisfied, killing an alias at launch with a 400.** openrouter died on every real turn with maximum context length is 262144 tokens. However, you requested about 265483 (33796 of text input, 103687 of tool input, 128000 in the output). Three separate defects converged:
 - limit.output was copied verbatim from models.dev, which is not internally consistent: **1099 of 5696 catalogued models report limit.output >= limit.context** (counted over raw published values, including the 104 rows whose context is 0 — the resolver later treats those as unknown rather than as a cap of zero, which is why the both-fields-positive count is 995), which is physically impossible — the output budget is carved out of the context, so it is strictly smaller. A record in that shape has a context-sized number sitting in its output slot (kilo’s row is the pure form: {context:262144, output:262144}).
 - models.dev stores limits per (provider, model) PAIR, and only openrouter’s row inflates this model to 1000000 while parking its true context (262144) in output. The same model id reads 262144 under nvidia, kilo and nano-gpt.
 - scripts/lib.sh exported the input guard **only when the context was ≤ 200000** — fail-open, so precisely the large-context providers got no guard at all. The pre-existing “output ≥ context ⇒ don’t export” branch was actively

harmful: declining to export hands the decision to Claude Code's own 128000 default. (The 128000 in the error above was this clamp, which coincidentally equals that default.)

`providers_resolve.py:derive_limits()` fixes this in two independent ways. First, the output cap is always **carved** out of the context rather than trusted verbatim, which by itself corrects every one of those 1099 rows — a separate output $\geq$ context detector was written for them and later removed once it was proven to change the result on **0 of 5696 rows**: whenever output $\geq$ context the carve already yields $\text{cap} < \text{context} \leq \text{output}$, so $\min(\text{output}, \text{cap}) == \text{cap}$ regardless. Second, a `:free` record claiming a larger output budget than its paid sibling now raises only a *suspicion*, which is adjudicated against the rest of the catalog: a context is lowered only when at least **three distinct providers, not counting the accused record's own provider**, publishing the same model (matched after normalising `:free` suffixes, vendor prefixes and case, and gated so an unrelated vendor's same-named model cannot vote) contradict it — and only to a value one of them actually publishes. Both halves of that rule are load-bearing and were learned the hard way: while the accused counted itself and the threshold was two, a single peer decided every verdict, because at two voters the lower median is the minimum. That cut a genuine 1,000,000-token window to 8,192 under a note claiming two independent providers had agreed, when nothing had agreed with anything. Adjudication is further restricted to records where $\text{output} < \text{context}$; where the two are equal the record shows the catalog's commonest mislabel — output copied *from* context — which the carve already corrects, and comparing that copy against a sibling's genuine output cap is a category error that produced three wrong reductions before it was caught. What this mechanism can establish is only that a claim is not credible **as data** — a plausibility ceiling drawn from peers, never a measurement. It cannot establish "the model's real context", because there is no such quantity: each (provider, model) pair is its own deployment, and llama-3.2-3b-instruct is genuinely served at 16000, 32768, 80000 and 131072 by different hosts with none of them wrong. It therefore has a permanent, deliberate blind spot in the opposite direction: a genuinely throttled `:free` tier is corroborated at its paid siblings' value and left alone. The earlier single-sibling version was wrong on half its live firings — it collapsed nemotron-3-ultra-550b:free from 1,000,000 to 65,536 (93.4% of the window lost) and gemma-4-26b:free from 262,144 to 32,768, even though NVIDIA's own record and 14 of 15 providers respectively publish the larger value. Corroboration restores both while preserving the two genuine catches. The guiding asymmetry — understating a window only costs capability, whereas overstating one kills the alias at

launch — is bounded by a second one learned here: an 8192-token output cap on a coding model is itself a dead alias, so a detector that fires wrongly is not made safe by erring small. A large but credible record does not trip either detector: xiaomi's mimo-v2.5-pro genuinely serves {context:1048576, output:131072}, and its **context is not collapsed** — it derives 1048576 / 128000, where the output is reduced only by the separate, pre-existing clamp to Claude Code's 128000 ceiling, not by the mislabel detectors. The guards are now **co-derived** in scripts/lib.sh — window = min(context - output, 200000), exported for every known context — which closes both the fail-open gate and the 200k-270k dead zone in which a real window got no input guard. openrouter/kilo now derive 262144 / 102144, leaving 22517 tokens of headroom. **This defect predates v1.24.0**: HEAD's resolver produced 1048576/1048576 for openrouter — limits belonging to a different model entirely — so the credit-aware pin re-derivation already improved it.

- **A concurrency race could destroy the user's alias file on a clean run.** cma_ensure_alias_file performed six sequential temp+rename migrations plus two direct cat >> "\$ALIAS_FILE" appends, racing cma_provider_write_alias — which claude-providers list --refresh-aliases fires **21× on every shell start** via the session hook. No lock existed on the alias file anywhere. Each writer was atomic and preserving *in isolation*, which is exactly why no single-function bug could be found: the second writer's read predates the first writer's rename, so it faithfully writes back a stale whole file. This was observed live — the file went 37288 → 282 → 32112 → 974 → 909 → 25682 → 130 bytes within four seconds, losing both wrapper functions and all four claude1..4 account aliases, leaving every alias pointing at an undefined function. It is **not** interrupt-only: the corruption timestamp preceded the suspected SIGTERM by ~14 minutes. All alias-file writes now funnel through a single committer (cma_alias_commit): the COMPLETE file (header + both wrappers + every account alias + every provider alias) is rendered into one temp and published with a single mv, behind an exclusive lock (flock(1), falling back to an atomic-mkdir lock with rename-verified stale breaking for hosts without flock, e.g. macOS). A byte-identical no-op guard renders and compares *before* taking the lock, so a settled host performs **zero renames per shell start** and the common path cannot enter the race at all; the session hook additionally pins a zero-wait acquire so it can never block a shell. INT/ TERM are masked across the critical section, and a sanity gate (header, both wrappers, the exact alias-name set, an account-alias floor) parks a rejected candidate as <alias-file>.rejected.<ts> rather than publishing it. The render-once design also retires the drop-and-re-append migrations that leaked ~15 orphaned # Wrapper: comment blocks and reordered the file. Covered by scripts/tests/test_alias_file_concurrency.sh (69 assertions). Teeth: against the pre-fix code the storm test records **115**

structurally broken states published during a single storm — a 35KB file collapsed to 1274 bytes with both wrappers gone, reproducing the live incident — and reports 16 failed, 17 passed; after the fix, three consecutive runs give 69 passed, 0 failed with an identical 35257-byte result (the storm case's three runs were 37 passed when first landed; the file has since grown its lock-release / gate-integration / account-floor coverage). `test_lib.sh` now also lints that exactly **one** code path writes or renames onto `$ALIAS_FILE`.

- **Unknown or partial model limits failed open.** A pinned model absent from the catalog produced `context_limit=None`, `max_output=None`, so neither guard was exported and Claude Code fell back to its own 128000 default against an unmeasured window — the same fail-open class as the gate above. inference shipped verified and launchable in exactly that state. Unknown limits now resolve to a conservative *number* rather than to silence: `UNKNOWN_MODEL_CONTEXT = 128000` (measured, not invented — **93.95%** of the **4544** tool-call-capable models in the live catalog publish ≥ 128000 , and `p10 = 128000`), narrowed further by the provider's own `p10` over its tool-call-capable models, with `output < context` enforced in a single place. A bare provider ceiling was not enough. Measured as *"how often does this mechanism pull the fallback below the 128000 default"*: the bare ceiling (published = `pool[-1]`, all the old mechanism was) did so for only 3 of 167 providers (1.8%) — inference 125000, `llmtr` 16384, `morph` 32000 — leaving the default free to overstate a small window (pin an uncatalogued 32k model on a provider whose ceiling is 10,000,000 and the wrapper would have exported a ~120k input window against it). The per-provider percentile does so for 32 of 167 (19.2%) instead — both counts unaffected by the median clamp: the ceiling expression reads no floor term at all, and the clamp moves only two estimates (inference 65536→16000, `atomic-chat` 65536→32768), both already below 128000 either way. It carries a `MIN_USABLE_UNKNOWN_CONTEXT` floor of 65536 because an unfloored percentile swings the other way (poe's raw `p10` is 480 tokens, which would trade a 400 for an unusable alias). (A different question — "is `pool[-1]` the term that decides the returned value" — answers 1 of 167 under the shipped clamp and 2 before it; that is the reading the old "2 of 167" was computed under.) Operator overrides in `providers/overrides.json` are now validated rather than trusted: `true`, `-1`, `0`, `1` and `50000.5` each previously produced a state in which the launch wrapper exported **neither** guard — silently reproducing the exact fail-open being closed — and a quoted "200000" was dropped without a word. Non-positive, boolean and fractional values are rejected with the reason recorded, clean digit strings are honoured, and an override that cannot carve a usable cap rolls back to the derived pair. Catalog rows with `"output": 0` (166 rows) or `"context": 0` (104 rows) are treated as unknown rather than as a binding cap of zero; this removes the resolver's silent dependence on the launch

wrapper re-deriving a cap on its behalf. A launch-gate refusal was deliberately rejected as the remedy: it would destroy a proven-working alias, and the operator's realistic response is --force, which bypasses the gate *and still exports no guards*. This also surfaced a second defect: the MIN_SAFE_OUTPUT floor was applied unconditionally, so any window below it came out

inverted — 136 of 5696 catalogued rows emitted

output >= context (mistral/open-mistral-7b at 8000/8000 → 8192). *Method: replay the pre-fix carve (cap = max(min(ctx - 160000, 128000), 8192), no halving branch) over every catalog row and count rows whose emitted output (min(published_output, cap)) is ≥ their context.* Now 0 violations and 0 missing contexts across all 5696. inference derives 125000/8192, poe 128000/8192 (a latent instance — its stored limits were stale and the next sync would have emptied them), and kimi-for-coding 262144/102144. Teeth: 14 failed, 329 passed before, 343 passed, 0 failed after.

- **Every router-transport provider alias failed at launch.**

The Go router implemented start|ui|serve|web|stop|config|help and no agent-launch subcommand, so the wrapper's ccr default-claude-code -- "\$@" (in scripts/lib.sh's cma_run_provider) fell through to the dispatch default: branch, printed Profile "default-claude-code" was not found or is disabled. and exited 1. The binary's own --help advertised the grammar it did not implement. The router now implements the agent-launch subcommand (cmd/ccr/launch.go, dispatched as default-claude-code/code).

- **ccr restart was also unimplemented, and failed silently.**

The wrapper invoked it under >/dev/null 2>&1 || true, so the missing subcommand produced no error anywhere. (That swallowing || true no longer exists — it was removed later in this same release, once a review showed a failed restart leaves the gateway serving its previous route while the config file reads back correct; the launch now refuses rather than proceeding against a route it cannot show was applied.) Because a running gateway keeps serving its startup config (cmd/ccr/serve.go:137-143), the per-launch config rewrite was never applied: an alias would route to whichever provider the daemon happened to start with, silently and with no diagnostic. restart is now implemented and replays the original flags.

- **The LAUNCH path could silently rebuild an authenticated gateway UNAUTHENTICATED.** ensureGateway has three arms: a live gateway that answers (use it), a live service whose gateway does not answer (bounce it), and nothing running (autostart). The middle arm and cmdRestart both refuse when the recorded service had inbound auth enabled and no CCR_API_KEYS is visible to the call (cmd/ccr/launch.go:160, cmd/ccr/service.go:386) — the autostart arm carried no such refusal. (The launch subcommand — cmd/ccr/launch.go — is itself new in this release, so this contrasts arms within a file authored here, not a previously shipped omission.)

That arm is also the one reached when a previously-authenticated service *died* leaving its pidfile behind: readServiceState succeeds and still reports AuthEnabled: true, but processAlive is false. A launch from a shell without CCR_API_KEYS therefore brought the gateway back with inbound authentication silently off — startService hands its key list to applyChildAPIKeyEnv, which *unsets* CCR_API_KEYS when the list is empty. The autostart branch now mirrors the refusal (err == nil && st.AuthEnabled && len(flags.APIKeys) == 0, cmd/ccr/launch.go:199) and names the risk in cmdRestart's own word, UNAUTHENTICATED; a genuinely absent pidfile recorded no auth posture, so a first-ever fresh start is unaffected. Teeth: cmd/ccr/launch_authfailsafe_test.go constructs exactly that state (a reaped pid, plus a stale AuthEnabled pidfile) and asserts the refusal names UNAUTHENTICATED. Neutering the refusal makes it FAIL — without it ensureGateway proceeds to rebuild, and what comes back is a gatewayReadyTimeout (15s, launch.go:68) readiness error, never the asserted refusal; with the refusal in place the test passes in 0.008s. **Bounded honestly:** the gateway binds loopback and inbound auth is opt-in, so the blast radius is a local downgrade, not an exposed endpoint. It is still a real silent downgrade of a posture the operator explicitly turned on, which is precisely the class cmdRestart already refused. (Nit landed in the same change: cmdLaunch's unused profile parameter is now _.)

- **Gateway address was not recorded, and a gateway-disabled service stayed disabled across restarts.** service.json carried no gateway address, so restart could not replay it and the launcher could not find it. The pidfile now records gatewayHost/gatewayPort (older pidfiles backfill to the documented defaults rather than host ""/port 0), and on finding a live service whose gateway does not answer, the launcher restarts it with the gateway enabled.
- **The test suite destroyed two production scripts in the working tree.** scripts/claude-session.sh (201 lines) and scripts/claude-sync-state.sh (103 lines) were found truncated to 8-line and 2-line test stubs. Cause: tests wrote stubs to \$HOME/.local/bin/claude-session, which in the REAL home is a symlink into this repo (install.sh links every claude-*.sh there), and cat > follows symlinks — so any run without an effective sandbox HOME overwrote the production script through the link. Symptom: every alias failed with No conversation found with session ID: 11111111-2222-3333-4444-555555555555. Both files restored from git. Hardened in scripts/tests/lib/sandbox.sh: a new sandbox_stub helper (asserts the sandbox, asserts the target is inside it, and **removes** an existing symlink instead of writing through it) plus an assert_sandboxed guard that aborts with exit 99 if \$HOME is not a mktemp sandbox, called from make_sandbox itself. Every stub write across the suite was converted to it.
- **9 of 13 pinned providers carried the wrong token limits.** When providers/overrides.json pinned a strong_model/

fast_model, providers_resolve.py copied the pinned id but never re-derived the model's context/output limits — so a pinned model inherited whichever limits the auto-selection had picked first (e.g. nvidia inherited z-ai/glm-5.2's 1000000/131072 instead of the real 256000/65536), and the launch wrapper exported CLAUDE_CODE_MAX_OUTPUT_TOKENS at roughly 2× the model's true cap. A sibling bug ignored the documented context_limit/max_output override fields entirely. Both fixed at source (limits are re-derived from the model actually selected; explicit override fields are honoured), with regression tests; an unknown pinned model now yields honest None limits rather than another model's numbers. (nvidia's strong/fast slots were also un-inverted.)

- **Test-suite hygiene, three latent hazards fixed and guarded.** (1) A leaked Go module cache: because a sandbox is \$HOME, every Go-building test wrote a 500–850 MB read-only go/pkg/mod cache that cleanup_sandbox's rm -rf could not remove (EACCES), leaving ~35 GB of orphaned cma-test.* dirs; fixed with a chmod -R u+w before removal. (2) test_sessions.sh wrote fixed /tmp/cma-test-*.log paths outside the sandbox (cross-run collision) — moved under the sandbox. (3) test_install.sh called make_sandbox twice, orphaning the first sandbox and writing its rc/log outside the live one — fixed with a fresh_sandbox helper. (4) The live-providers leg now fails the suite **only** for providers that independently reached status verified; a provider already classified failed/unverified/orphaned account-side is reported on its own KNOWN-NON-WORKING line, never silently skipped and never counted as a pass — so a permanently-red gate can no longer hide a genuinely new regression.
- **_cma_mtime returned nothing usable on macOS, and the same broken idiom existed twice.** The idiom is date -r "\$path" +%s with a stat -c %Y fallback. Both halves fail on BSD/macOS for independent reasons: date -r there reads its argument as **epoch seconds**, not as a file path, and stat -c is GNU-only (BSD spells it stat -f %m). So on macOS the first branch silently produced a garbage/zero read and the second could not run at all. The two sites differ in provenance, and the distinction is load-bearing for what is claimed. _cma_mtime (scripts/lib.sh:279) and the whole rolling-rc-backup subsystem it feeds (cma_newest_rc_backup, cma_backup_rc_file, the <rc>.cma-orig / <rc>.cma-backup.<epoch> files) are **new in this release** — none of them exists in HEAD — so _cma_mtime returning 0 for every file, and thereby degrading cma_newest_rc_backup from “newest by mtime” to “last by name”, is an **intra-release defect caught before commit**, never a shipped one. The genuinely pre-existing defect is the second copy: the session-refresh hook lib.sh emits inside a <<'HOOK' heredoc (scripts/lib.sh:1492, and shipped today at HEAD's scripts/lib.sh:1416), which runs in the **user's** shell and cannot call _cma_mtime, so it has to stay self-contained. Both sites were fixed independently to the portable stat -c %Y ... || stat -f %m ... pair — GNU-first,

so on Linux `stat -c` succeeds and the BSD branch is never reached. A separate portable-mtime precedent already existed in-tree — `claude-providers.sh`'s case `"$(uname -s)" dispatch (Darwin*) stat -f %m / *) stat -c %Y`, lines 114-117) — but it is **not** the same idiom: its own comment (:112-113) records that an earlier `||` chain in the *reverse* order broke on Linux, because there `stat -f` also succeeds. It is cited as prior art for handling both platforms, not as a line the fix was copied from. Guard: `scripts/tests/test_rc_safety.sh` (f) now lints `lib.sh` for the date `-r "$var" +%s` form and requires the helper to carry the BSD branch — **21 passed, 0 failed. Honest scope:** neither half was a data-loss defect. For the new-in-this-release subsystem, its data-safety never rested on the mtime read at all: rolling backups are named `<rc>.cma-backup.<epoch>` (`lib.sh:327`), so name-order and epoch-order coincide and the “newest” pick is correct regardless of mtime, and `<rc>.cma-orig`, the load-bearing recovery point, is written exactly once and never consulted by mtime — but that describes how the intra-release defect *would* have behaved had it shipped, not production history, because none of that code is in HEAD. In the one half that DID ship, the emitted hook, the failure was a missed TTL rather than a wrong one — its fallback is `echo "$now"`, so the computed age was 0 and the background `claude-providers sync` simply never fired. This was a correctness and portability defect, not a data-loss one, and is not claimed as more.

- **Concurrent suite runs are now serialized** (`scripts/tests/lib/suite-lock.sh`). Two overlapping `run-all.sh/run-proof.sh` invocations mutate the repo while it tests itself, which is what once made a set of deterministic tests look flaky; the lock (`flock`, with an `atomic-mkdir` fallback for macOS) is re-entrant for the nested `run-proof.sh` → `run-all.sh` case and bounded-waits then exits 75 on contention. It caught a real second session running the suite during its own verification.

Fixed — the verification that was supposed to catch this

- **A layer-4 PASS could be non-attributable — it proved *so me* backend worked, not the one under test.** Every router-transport provider rewrites `ccr's` shared `Router.default` to itself immediately before launching, but an alias whose `base_url` IS the gateway (`http://127.0.0.1:3456/v1`) trips a self-reference guard in `cma_run_provider` and skips that rewrite — so it inherited whatever the *previous* provider left in `~/.claude-code-router/config.json`. `helixagent` was badged verified on a turn served by a different provider entirely: 157,419 tokens passed through a nominally 24,576-token alias because it had inherited a ~1M-context route. The evidence file recorded the request and the response but never *which backend served it*, so a pass and a pass-by-inheritance were indistinguishable —

which is why this survived a release. (The `modelUsage: claude-opus-4-8[1m]` line in every layer-4 file is not a leak to the operator's Anthropic account: the router branch never exports `ANTHROPIC_MODEL`, so Claude Code labels and prices turns with its own defaults while ccr rewrites the model server-side. It does mean that field can never be used to attribute a backend.)

`verify_superpowers_tui.sh` now records `# ROUTE-INTENDED:` and `# ROUTE-RESOLVED:` in every evidence file and fails the leg with `# FAIL: route-mismatch` when they disagree, or `# FAIL: route-unknown` when the route cannot be read — an unattributable turn now fails instead of passing. The gate does not merely compare the on-disk config, because a written config that was never *applied* proves nothing: `ccr restart` ran under `|| true`, and `cmdRestart` genuinely refuses to bounce an authenticated gateway when `CCR_API_KEYS` is not visible (`cmd/ccr/service.go:385-390` returns 1), so a swallowed failure would leave the gateway serving its previous route while the file read back as correct. The router exposes no live-route query (`/health` reports a provider *count*, not a route), so the gate instead requires a **restart receipt** bracketing the launch — a new gateway listening on line in `~/.claude-code-router/service.log`, or a changed `service.json` — and fails closed with `# FAIL: route-unproven` when it cannot show the route was applied. Both `.Router.default` **and** `.Router.background` are compared (a partly-foreign turn emits `# FAIL: route-mismatch-background`), and `jq` is now a hard precondition rather than a silent skip, since a host without it cannot run router providers at all. Route failures are evaluated independently of provider status: a rejected key explains a provider that cannot answer, but nothing about an account explains evidence attributed to the wrong backend. **The guarantee, stated precisely:** a layer-4 PASS on router transport is impossible unless both router keys name the alias under test AND a restart receipt brackets that launch. Two limits remain, and are not claimed away: the receipt brackets the whole launch rather than the individual request (the suite lock, not the gate, excludes a concurrent rewrite), and it proves *a* config load rather than that the loaded bytes were the ones read back — closing that would need a live-route query the router does not expose. Covered by `scripts/tests/test_layer4_route_attribution.sh` (66 assertions across 10 legs). Teeth are proven per-finding by mutation — reverting any one fix flips exactly its own assertions — because HEAD is not a usable baseline here (it predates the whole feature). The original bluff is reproduced verbatim in the mismatch case: a `# PASS` written for a turn served by chutes while the alias under test was `routertest`. The previously vacuous assertions were replaced with ones that execute the production sweep; against a mutant with that sweep deleted, the old assertions still passed and the new ones fail.

- **v1.23.0's "run-proof ALL GREEN" and "10-11 aliases verified live" were false claims.** `scripts/tests/`

verify_providers_live.sh's layer-4 handler called _pass on a layer-4 **FAIL** — it asserted “the verifier ran”, not “it passed”. 21 alias failures were counted as passes and the leg reported “40 passed, 0 failed” while the router launch was broken end-to-end. Compounding it: no gate anywhere grepped the proof directory for # FAIL: markers, so 37 stale markers sat on disk unread. And the legs that produced the “10-11 verified” figure (verify_aliases_live.sh, alias_e2e_test.py) never launch an alias at all — they curl provider endpoints directly, explicitly “bypassing ccr”, so they could not have detected a broken launch path under any circumstances. Fixes: line 126 is now _fail; the layer-3 unverified branch likewise (providers-semantic.sh emits it only on a definitive driver failure — transient conditions already route to skip); and a new proof-directory sweep gate fails the run if any evidence file **from the current run** ends in # FAIL:, independent of the stdout classification.

- **Layer 3 could de-verify a working provider on its own completion budget.** chatComplete returned the model's content and discarded finish_reason, so a reply severed mid-generation was indistinguishable from a model that had not complied. Measured live: siliconflow's round-1 reply came back as ZETA-9-ORANGE- against the sentinel ZETA-9-ORANGE-7f3a — a strict prefix, cut mid-token — and was reported as sentinel not found in response, a definitive exit-1 that demoted a provider whose layer-4 TUI turn PASSED. It now returns chatOutcome{Content, FinishReason}, and a round-1 sentinel no-match at finish_reason == "length" is classified **infra** (exit 3 = honest SKIP, never a demotion): a truncated probe never completed, so it yields no verdict about the model at all, and blaming the model for a budget we chose is exactly the bluff this layer exists to avoid. Reading that path surfaced a second, latent defect. The judge branch states its own policy in comment — “a broken judge must never demote the model-under-test” — and routes every judge *call* failure to failInfra, but an unparseable judge *reply* fell through to fail() (exit 1) even when the reply was severed at finish_reason == "length". A judge that merely ran out of tokens therefore de-verified the model under test, contradicting the judge-branch policy stated directly above it in its own comment; that case now routes to failInfra too. The budgets were then raised — round1MaxTokens 256 → 2048, judgeMaxTokens 64 → 2048, and round 2's bare literal 512 → the named round2MaxTokens = 2048 — and the three sites differ **in kind**, which is the point:
 - **The judge at 64 was the active defect.** Measured on deepseek-v4-pro, 3 samples per budget, task = emit one integer: reasoning 78-361 tokens; 64 = **0/3**, every sample finish_reason=length with empty content — the exact truncation shape above. 128 = 0/3, 256 = 1/3, 512 = 3/3. 512 is nonetheless the wrong fix: it passed only as a small-n artifact, and widening to 9 samples surfaced a 500-token

completion, leaving 12 tokens of headroom. 2048 is chosen with margin rather than fitted to the sample.

- **Round 1 at 256 was NOT broken, and no claim is made that it was.** Same model, task = echo an 18-char sentinel: reasoning 58-97 tokens; $64 \Rightarrow 0/3$, $128 \Rightarrow 2/3$, **256 $\Rightarrow 3/3$** . It is raised for a different and more interesting reason: **reasoning-token distributions do not transfer across models, or across tasks.** siliconflow truncated at 256 on that same round-1 task — the severed-prefix case above. A per-task budget tuned on one model silently de-verifies another. And truncation at *either* round de-verifies a working provider — round 1 severs the **sentinel** (a mechanical exact-match check, exit-1 on no-match), round 2 severs the **description** (scored by the judge, which fairly rates a severed or empty description below threshold 2 $\Rightarrow$ fail() $\Rightarrow$ exit-1); the *only* round-2 path that is a mere skip is a truncated judge *call/reply*, which D1 routes to failInfra. After D1 both truncation sites are prevented by budget rather than one being intercepted, which is why **both** budgets were raised, not just round 1's.
- **Round 2 at 512 was the release's second counted blocker — it de-verified inference.** The round-2 *description* call (glm-5.2, max_tokens the only variable) came back finish_reason=length with empty content on two of three samples at 512; the judge fairly scored the severed/empty description below threshold 2 $\Rightarrow$ fail() $\Rightarrow$ **exit-1**, demoting a provider whose layer-4 TUI turn PASSES. This is the symmetric twin of the siliconflow case, not a bonus clause: **both** run-proof failures were D1 truncation de-verifications — siliconflow's **sentinel** severed in round 1 (fixed by round1MaxTokens), inference's **description** severed to empty in round 2 (fixed by round2MaxTokens). Raised to 2048 the description completes (finish_reason=stop on all three samples, well inside the budget) and 12/12 verifier runs return verified, judge scores 2-3, never below threshold — though six of the twelve sit at exactly 2, a thin-but-consistent margin, not a proven bound. **Honest bound:** D1 does not add a round-2-description $\rightarrow$ infra branch; it gives the description enough budget to finish. A description that genuinely needed >2048 would still be judged low and exit-1 — the fix is “enough budget to finish,” not “truncation can no longer de-verify.” Two scope limits, stated rather than papered over. The **default** judge is Groq llama-3.1-8b-instant (scripts/providers/judge.env.template), a non-reasoning model, so the judge=64 defect was **latent in the default configuration** — it bites only when a reasoning model is configured as the judge. And the measurements are one model, one provider, one prompt pair: a sample maximum is not a distributional ceiling (the 500-token judge sample appeared only when n went 3 $\rightarrow$ 9), so these budgets clear the observed tail with headroom rather than claiming a proven bound. Landed in submodules/

LLMsVerifier/llm-verifier/cmd/semantic-code-visibility/main.go; covered by that package's main_truncation_test.go (6 tests: budget truncation named as such, a genuinely-empty reply still reported as empty, finish_reason surfaced to the caller, and a budget floor asserted per site — **not** a uniform one: round2MaxTokens $\geq$ 2048 and judgeMaxTokens $\geq$ 1024, but round1MaxTokens $\geq$ 512, so round 1 could be lowered to 512 and the suite would still pass — a softer guard than this entry's own argument about round-1 sentinel truncation implies) — go test ./cmd/semantic-code-visibility/ passes.

Added — regression guards

- **scripts/tests/test_ccr_conformance.sh** — static, no-network conformance between the launch grammar the toolkit speaks and the grammar the bundled Go router understands. Scans scripts/lib.sh for ccr <word> in command position, parses the case arms of the router's top-level dispatch, and asserts required $\subseteq$ supported. This would have caught the regression with no network and no launch.
- **ccr test stubs now fail loudly on an unrecognised subcommand** instead of exit 0. The router path's only coverage was fake stubs that silently succeeded on anything they did not recognise — a stub can certify nothing but the stub.
- **TestMain guard in the router's test package** (cmd/ccr/main_testmain_test.go) prevents tests from spawning the real service binary. startService spawns a detached child from os.Executable(), which under go test is the test binary; the child re-ran the entire suite and forked exponentially, leaving 600+ live processes until fork(2) returned EAGAIN and unrelated tests failed with "resource temporarily unavailable". Tests that legitimately need a service process must go through withStubService(t).
- New guards, each proven to fail when the thing it guards is broken (mutation tested): test_mutation_residue.sh (refuses to ship a && false / always-pass mutation left behind by testing), test_providers_gate.sh (a verified provider's failure still fails the suite; account-dead ones do not — with an anti-vacuous guard that the gate genuinely discriminates), test_sandbox_hygiene.sh (no test writes a fixed /tmp path or a bare redirect into .local/bin), test_sandbox_leak.sh (a test that leaks a sandbox fails even though it exits 0), test_suite_lock.sh (34 assertions, 3 mutations killed), and test_provider_credit.sh (127 assertions for the credit-tier rule).
- The layer-4 superpowers-engagement check was replaced: the old fuzzy vocabulary grep produced both false passes (a confabulated claim) and false failures (a genuine engagement phrased differently). It now poses an unforgeable secret-knowledge challenge — one exact cell of the skill's own table,

content a model can only produce by having loaded the skill — plus a distinct empty-result verdict for a model that runs but emits no text.

Verified

Credit-aware selection (this release's new capability): - scripts/tests/test_providers.sh **405/0**; test_provider_credit.sh **127/0**. - The two rule-encoding branches are **mutation-proven** to have teeth: breaking the unknown→free conservative default makes the anti-vacuous guard fire (available and unknown stop differing); breaking the credit→paid branch makes the paid models unreachable (19 assertions fail). Both restored byte-identical. - LLMsVerifier credit/cost-aware code: go build ./... clean; the providers, scoring and selection packages test green.

Launch-regression fixes (carried from the v1.24.0 work): - **Live, through the real alias path — final run-proof.sh (2026-07-21)**: all six legs rc=0 (sandbox · live · providers · aliases · alias-e2e · constitution). The aliases leg records PASS: 9 FAIL: 0 SKIP-QUOTA: 0 SKIP-TRANSIENT: 0 SKIP-GATED: 12 TOTAL: 21, and the providers leg 9 passed, 0 failed, 12 gated-skipped. The 12 gated-skipped are account-dead providers (funds/keys) the verification gate correctly filters out — they are not claimed to work. helixagent did change from its earlier PASS to the expected route-attribution de-verification, exactly as predicted; that is the correct outcome, not a regression. The two layer-3 truncation fixes are proven in the regenerated evidence: providers-siliconflow-semantic.txt and providers-inference-semantic.txt now both read verified (round-1 sentinel intact, round-2 judge scored 3 and 2), where before this release they read unverified on severed responses. - **Evidence provenance**. Only proof artefacts produced after 2026-07-20 13:54 are cited: from that point ~/.claude-code-router/service.log retains the gateway restart records that let each launch be attributed to the route it actually owned. Older artefacts (Jul 5/18/19) predate both those retained records and the switch to the Go router as sole router, cannot be attributed retroactively, and are deliberately not relied on for any claim here.

- **Full hermetic suite (2026-07-21): 42 files / 42 passed / 0 failed — ALL GREEN**, run against the final tree (run-all.sh, which auto-discovers every test_*.sh, including the new test_redact.sh and test_semantic_evidence.sh). This is the real tally the earlier “NOT yet asserted” note deferred to: a prior 36-file run predated these fixes and covered fewer files, so it was deliberately never claimed as a full-suite green — claiming it would have repeated the v1.23.0 error this release exists to correct.
- **Independent whole-branch review (2026-07-20): GO** — on the tree as it stood that day, CHANGELOG honesty, docs-vs-

code fidelity, the I-2 fix, and the §11.4.120 test reconciliation were all verified with pasted evidence, and the sole flagged blocker was the mechanical submodule-commit step (addressed at release time), not a code defect. **Scope — this GO covers only what the review saw, and is not a sign-off on the shipped tree.** Substantial work landed after it: the alias-file render-once race fix, the route-attribution gate hardening, the empty-/unknown-limits fix, the rc-safety + `_cma_mtime` work, the layer-3 D1 truncation fix, the semantic fail-open fix, and these CHANGELOG entries themselves. Under this project's own re-review discipline, changes after a GO re-arm the review, so the 2026-07-20 result is recorded as what it was, a green light on an earlier state, not a release sign-off.

- **Final whole-branch review (2026-07-21): GO on the shipped tree.** The two thinnest-covered pieces — the `providers-semantic.sh` evidence mirror and the layer-3 D1 truncation routing — were verified verdict-safe, secret-safe (the mirror lands in an evidence file that is `_redact`-ed before commit), and regression-safe (every new branch is FAIL→SKIP, never FAIL→PASS, so no non-compliant model can newly pass; `go vet / go test / go build clean`). Mutation residue and secrets are clean across all three repos, and docs name only symbols that exist in the code. This is the sign-off the 2026-07-20 result could not give.

Scope of what “ALL GREEN” claims here: the hermetic suite and all six `run-proof.sh` legs pass on the final tree, and the 9 verified provider aliases work live end-to-end. It does NOT claim the 12 account-dead aliases work — those are gated out for lack of funds/keys, reported as known-non-working, and never counted as a pass. The release is cut from this state.

v1.23.0 — 2026-07-20 — Bundled Go claude-code-router is now the SOLE router (JS fully replaced) + full retest

The vendored Go `claude-code-router` (submodule, installed as `ccr` via `claude-ccr-build`) is now the toolkit's one and only router; the original Node `@musistudio/claude-code-router` is no longer preferred, required, or the advertised fallback.

Changed

- **Go fully replaces JS.** There was never any functional code requiring the Node router — detection (`command -v ccr`) and the identity-guard grammar check (`ccr start/ccr serve`) are router-agnostic and already pass the Go binary. This release removes the remaining JS advertising: the missing-`ccr` hint (`lib.sh`) and

the foreign-ccr refusal now point solely at claude-ccr-build (the bundled Go build) and no longer name @musistudio/claude-code-router or suggest npm install; the claude-ccr-build Go-toolchain-missing hint demotes the Node router to a last-resort note; install.sh's warning drops the JS clause. A new migration marker regenerates already-installed alias files so existing users pick up the reworded guard.

test_output_tokens.sh now asserts the refusal points at claude-ccr-build AND no longer advertises the JS npm router.

Fixed

- **test_lib.sh proof-secret scanner: token-boundary anchoring.** The proof-dir secret scanner matched a key prefix embedded mid-identifier (re_ inside a Go-module-cache path retire_connection_id_frame_test.go captured as build noise), a false positive. Prefixes now match only at a token boundary (line-start or a non-word char before them), so a real leaked key (after a quote/=/:/space) is still caught while innocent identifiers are not. Pinned by a fixture regression guard (embedded prefix NOT flagged, real sk-ant- key IS flagged).
- **Bundled LLMsVerifier submodule: data race fixed** (submodule commit f9b875cf). InMemoryContinuousEvaluator.executeRun wrote run.Status on the nil-debateEval path outside the mutex while GetRun read it concurrently — a race the -race detector flagged. The write moved inside the lock; the whole module now passes go test -race ./..., pinned by a new concurrent-stress regression guard.

Verified (full live retest — captured evidence in scripts/tests/proof/)

- Hermetic suite run-all.sh (29 files) — ALL GREEN; run-proof.sh (6 legs) — ALL GREEN; verify_ccr_live.sh — 49/0; verify_helixagent_test.sh — 46/46.
- Challenge bank (submodules/challenges): go test -race + meta-runner (25/25) GREEN. containers submodule: build+vet+-race GREEN.
- **Live provider aliases** (api_keys.sh present): claude-providers sync + the run-proof alias legs verify **10-11 aliases LIVE** end-to-end (chutes, deepseek, helixagent, inference, kilo, opencode, openrouter, poe, siliconflow, xiaomi; nvidia works at HTTP 200 but the strict 2-probe verify intermittently hits free-tier rate-limiting). The remaining providers are **honestly gated — every failure is account-side, not a toolkit bug**: direct probes captured 401 (key rejected — mappings verified correct), 402/403 (no funds / suspended / quota), 429 (fair-usage rate-limit / insufficient balance). Evidence: scripts/tests/proof/60-provider-triage.txt. claude-providers prune

removed 3 stale status-only orphans. The launch gate refuses every non-verified alias — the anti-bluff design working as intended.

- The qa-all control-plane/anti-bluff legs that require a live Helix cluster, the host suspend-guard, or go-mutesting are honest host/tooling preconditions on this machine, not toolkit-code failures.

**v1.22.9 — 2026-07-20 — bundled
claude-code-router v0.4.9 (--upstream-
timeout) + verify_ccr_live.sh auth+proxy
live-proof legs (49 checks)**

**v1.22.8 — 2026-07-20 — bundled
claude-code-router v0.4.8
(authenticated outbound proxy config
block, redacted password)**

**v1.22.7 — 2026-07-20 — bundled
claude-code-router v0.4.7 (inbound
auth switch, --max-attempts, start/ui flag
forwarding)**

**v1.22.6 — 2026-07-20 — bundled
claude-code-router v0.4.6 (docs
correction; no behavior change)**

**v1.22.5 — 2026-07-20 — bundled
claude-code-router v0.4.5 (OpenAI-
facade long-context routing symmetry)**

**v1.22.4 — 2026-07-20 — bundled
claude-code-router v0.4.4 (streaming
token accounting on both relay paths)**

**v1.22.3 — 2026-07-20 — bundled
claude-code-router v0.4.3 (TLS/HTTP3
CLI flags, OpenAI-facade metric parity,
synchronous bind) + verify_ccr_live.sh
TLS/HTTP3 live-proof leg**

**v1.22.2 — 2026-07-19 — bundled
claude-code-router v0.4.2 (transport/
hot-reload/load live suites)**

**v1.22.1 — 2026-07-19 — bundled
claude-code-router v0.4.1 (metrics
polish + live e2e); toolkit
verify_ccr_live.sh live proof of the
bundled Go ccr**

**v1.22.0 — 2026-07-19 — bundled
claude-code-router v0.4.0 (/metrics +
semantic cache wired, Router.think,
exhaustive tests)**

**v1.21.0 — 2026-07-19 — bundled Go
claude-code-router built+installed as
ccr (claude-ccr-build); submodule
bumped to v0.3.0 (response cache +
cross-provider fallback wired)**

**v1.20.0 — 2026-07-19 — bump bundled
claude-code-router to v0.2.0 (multi-
protocol gateway: OpenAI inbound
facade, Anthropic passthrough,
classifiers, Think/LongContext routing,
hot-reload, redacted logging)**

**v1.19.0 — 2026-07-19 — port
deepseek+xiaomi to router (ccr)
transport + IPv4 fix**

v1.18.0 — 2026-07-19 — HelixAgent/ HelixLLM local-model exposure + 128k output-token clamp + ccr launch fixes

Added

- **HelixAgent/HelixLLM provider — a local llama.cpp model exposed as a first-class provider alias.** A new helixagent alias routes through the claude-code-router (ccr, :3456) to a local llama.cpp server (:18434) serving a Qwen3-Coder-30B gguf, presented to Claude Code as **Provider = HelixAgent / Model = HelixLLM** — the raw local endpoint is never exposed as its own catalogue entry, only via the HelixAgent/HelixLLM facade. `claude-providers list` shows helixagent verified HelixAgent/HelixLLM. **Live-proven (2026-07-19):** a completion through HelixAgent/HelixLLM via :3456 returns from the local model (HELIX_ROUTE_OK, real token usage), and the `/v1/models` catalogue exposes the local model only as HelixAgent/HelixLLM (the raw gguf path is not a catalogue entry). Known residual: the completion response's `.model` field still echoes the raw gguf path — the ccr catalogue and `claude-providers` surfaces are clean; hardening that response echo is a tracked follow-up.
 - **ccr v3.0.6 target-adapter registration.** The provider id / name / `models[0]` are set to HelixAgent / HelixLLM (id == name) in the live `config.sqlite app_config` so ccr's target-adapter registry resolves a routable adapter — the synthesized `provider-helixllm-*` id had mismatched the routing name ("Target adapter is not registered for provider"). Reload via the gateway `restartGateway` RPC (graceful, daemon stays up).
 - **Facade stability across claude-providers sync:** `detect_helixagent_record()` in `claude-providers.sh` + non-secret provider pins (`scripts/providers/helixagent.json`: base :3456/v1, model HelixAgent/HelixLLM, context 24576).
 - **ccr-self-loop guard (`_cma_ccr_self`).** The HelixAgent facade points ccr at its own :3456 gateway; the guard prevents an infinite self-referential launch loop.

Fixed

- **128k output-token clamp for every provider alias, on both transports.** `CLAUDE_CODE_MAX_OUTPUT_TOKENS` is clamped to ≤ 128000 for every provider alias on the native AND router transports (deepseek 384000 / xiaomi 131072 $\rightarrow$ 128000) via a single unified export site preceded by an unconditional unset (no stale inheritance, no competing paths). Reconciled with the `output- $\geq$ -context skip guard` into one coherent block; covered by `test_128k_output_clamp.sh` (38 assertions green in the 27/27

suite) and independently reviewed (Fable) as strictly safer than either input — no code path exports > 128000; zero/non-numeric budgets no-export rather than exporting a raw value.

- **ccr launch-grammar fix.** ccr v3.0.6 renamed the launch subcommand; the provider-alias launcher now invokes `ccr default-claude-code -- "$@"` (was `ccr code`) so router aliases launch Claude Code correctly, with a migration marker so the fix redeploys into the installed alias file.
- **zsh fi fi parse fix** in the native-launch guard.

Testing / validation

- Full toolkit test suite **27/27 green** post-install (incl. `test_128k_output_clamp.sh`); the standalone `verify_helixagent_test.sh` verifier **46/46**.
- **Live retest (2026-07-19):** native `claude1-4` provider-env-isolated; 10 providers verified (incl. `helixagent`); `HelixAgent` → `HelixLLM` live route `HELIX_ROUTE_OK`; `claude-providers list` `helixagent` verified; `ambient ~/.claude/settings.json sha256` unchanged (no session hijack).
- **Merge review (Fable, xhigh) GO** on the main-integration reconciliation (control-needle-validated conflict-marker scan; migration-instrument provably-can-see).
- Note: v1.17.0 was documented in this changelog but never git-tagged; its changes land on main via the same merge and ship in this release.

v1.17.0 — 2026-07-18 — Cross-alias session & background-agent continuity

Fixed

- **A session left under one alias was invisible from every other alias.** Opening a project from `xiaomi`, leaving the session with work in progress, and then opening `deepseek` (or any other alias) showed nothing — two independent root causes, both fixed and live-proven:
 1. **Session resolution applied only to the native transport's bare launches.** Every router alias (`kimi-*`, `poe`, `openrouter`, `chutes...`) always opened a FRESH session, and `alias -p "..."` on any transport skipped resolution entirely (“explicit args win verbatim”). **Fix:** `_cma_session_flags` now runs before the transport split — both transports resume — and conversation args get `--resume <sid>` injected unless the user already chose a session (`--resume/--session-id/ --continue/-c/--fork-session`) or invoked a non-conversation subcommand (`agents`, `mcp`, `export`, ...). Injection uses the new `claude-session existing-id`, which returns a session only when one actually EXISTS

— the older latest-id falls back to a deterministic UUID for never-used projects, and resuming that fails hard with “No conversation found with session ID”. **Live proof:** xiaomi (native) created a session in a test project; kimi-k3 (router) and deepseek (native) both resumed the identical session id and answered from its memory.

2. **Background agents were registered per config dir.**

Claude Code’s background-agent registry (daemon/roster.json + dispatch/, plus the jobs/ store) was local to each alias dir, so an agent started under xiaomi was invisible to deepseek. **Fix:** daemon and jobs are now shared items (CMA_SHARED_ITEMS + unify’s SHARED_ITEMS, drift guard enforced), daemon/roster.json is **union-merged** by cma_union_rosters (newer updatedAt wins per worker — a per-file last-wins would drop other aliases’ workers), and cma_migrate_daemon_dirs_once merges every existing provider daemon dir into the shared store (roster content stashed before the backup move — the first cut of the migration unioned paths that had already moved), replaces it with the shared symlink, and is idempotent via a marker file. Live-verified: rosters from four provider dirs unioned into one shared registry; all provider daemon dirs are now symlinks.

Added

- **Testing:** scripts/tests/test_session_flags.sh (12 assertions — both transports resume on bare launch, -p injection, no double-injection, subcommand passthrough, empty-session case, migration trigger); test_unify.sh daemon section (roster union semantics, dir linking, jobs, provider-dir migration with backup + idempotency — 107 assertions in the file). Challenges/HelixQA: check 7 in provider_aliases_challenge.sh (shared items, union merge, flags placement, existing-id, live daemon symlinks — 23/23 PASS live) and bank case cma-pav-cross-alias-session-continuity (bank now 12 cases).

v1.16.0 — 2026-07-18 — Output-token cap for router providers

Fixed

- **“Claude’s response exceeded the 128000 output token maximum” on router providers.**
CLAUDE_CODE_MAX_OUTPUT_TOKENS was exported **only on the native transport path** — every router provider (kimi-k3, poe, openrouter, ...) launched with Claude Code’s generic default output cap (128000 for models it does not know), so long reasoning responses died with that API error. **Fix:** the cap is now exported for **both** transports, before the transport split

(`_cma_out_guard`), valued from the provider model's real `limit.output` (`CMA_PROVIDER_MAX_OUTPUT`, e.g. 131072 for k3). Live-verified on the real wrapper: a kimi-k3 router launch now carries `CLAUDE_CODE_MAX_OUTPUT_TOKENS=131072`.

- **Catalog-conflation guard:** models.dev sometimes reports `limit.output == limit.context` (the “output” number is really the context size) or an inflated `limit.context` (nvidia5's llama-3.2-11b-vision: catalog claims 1M context; the provider's own error says 131072). Exporting such a cap makes Claude Code request more completion tokens than the shared window allows → 400 maximum context length ... you requested N. The guard exports **only a genuinely separate output budget** (`output < context`). nvidia5 was removed (its catalog metadata is wrong on both axes; backed up — self-heals via `sync --multi`).
- **Wrapper migration now covers the output-cap fix** — `_cma_out_guard` was added to the `cma_run_provider` self-heal marker chain, so wrappers written before v1.16.0 regenerate automatically on the next `install.sh/sync/shell` start (otherwise hosts would keep the native-only export indefinitely).
- **e2e tool-call flake:** alias `e2e_test.py`'s tool probe gets the same one-retry policy as the layer-1 verifier (weak free models occasionally skip an instructed tool call — `opcode2` false-FAILED a proof leg).
- **Router launch prompted interactively on some shells.** The wrapper's `ccr-config mv` ran as a bare command; on hosts whose interactive shell aliases `mv='mv -i'` (Fedora/RHEL defaults) the alias launch stopped at `mv: overwrite '.../config.json'? and hung or died on redirected stdin. All 16 mv sites in lib.sh/emitted wrappers are now command mv -f (bypasses aliases and shell functions, forces the overwrite).`
- **Cryptic failure when another tool named ccr shadows the router.** On a host where `ccr` resolved to a different program (a CCS-style profile manager), `ccr code` meant “launch profile ‘code’” and the alias died with `Profile "code" was not found or is disabled`. The router path now verifies `ccr` version names the real `@musistudio/claude-code-router` and refuses early with an actionable message (fix `PATH` / remove the shadowing `ccr` / install command).

Added

- **Testing:** new hermetic suite `scripts/tests/test_output_tokens.sh` (8 assertions — native + router export, empty-limit case, guard-before-split structure, migration trigger). Challenges/HelixQA: new bank case `cma-pav-output-cap-both-transport` and Check 6 in `provider_aliases_challenge.sh` (16/16 PASS live). Full suite: 24/24 files ALL GREEN; 6-leg proof ALL GREEN.

v1.15.0 — 2026-07-18 — Full Kimi variant support (OAuth subscription models)

Added

- **Every Kimi model the OAuth subscription serves is now a launchable alias.** `detect_kimicode_record` discovers the served models live (`GET {base}/models` with the OAuth token, unioned with the `models.dev` catalog since the listing under-reports) and emits one provider record per model — the old code exposed a single hardcoded alias. On this host: `kimi-for-coding` (account default, “K2.7 Coding”), `kimi-for-coding-highspeed`, `kimi-k2p7` (Kimi 2.7), and **`kimi-k3` (Kimi 3 — 1M context, reasoning)**. Every record goes through the same strict sentinel + tool-calling + semantic verification as every other provider.
- **`kimi_proxy.py`** — moonshot-flavored schema normalizer routed under every `kimi-*` alias (new `<family>_proxy.py` discovery rule in the launch wrapper). Model `k3` rejects any tool whose parameters carries a `$ref` not starting with `#!/$defs/` (400 ... not a valid moonshot flavored json schema, reproduced live); Claude Code’s tool schemas trip exactly that. The proxy hoists `$defs+definitions`, rewrites foreign `$refs` by last segment, and guarantees `parameters.type/properties`. Live proof: direct request → 400, same request through the proxy → 200.
- **Launch-time OAuth token freshness** (`lib.sh` emitted wrapper). The OAuth token lives ~15 minutes, so the old sync-time snapshot 401’d by the next launch — the root of “kimi-for-coding works once, then dies”. Freshness order at every launch: unexpired live credentials file → CLI-triggered refresh (`kimi -p hi`) → token-file snapshot (last resort).
- **API-key paths for kimi.com coding** — `KIMI_API_KEY` (catalog env) and `ApiKey_Kimi` (new `key-aliases.json` entry) both resolve to the `kimi-for-coding` provider as a fallback for hosts without an OAuth session. OAuth subscription records take **precedence** over API-key records (`unique_by merge, detector first`) — the subscription is the priority path.
- **`sarvam_proxy.py`** — Sarvam compatibility proxy (same family-discovery mechanism). Three distinct runtime incompatibilities were root-caused and fixed, each reproduced live first: system/user message content arrays (400 ... Input should be a valid string — flattened to joined strings) and Claude Code’s 64000-token output default exceeding the starter tier’s 4096 cap (`max_tokens` now clamped, overridable via `SARVAM_MAX_OUTPUT_TOKENS`). Result: the sarvam alias went from guaranteed-400 at launch to a real Claude Code PASS.

- **Challenges/HelixQA:** three new bank cases (cma-pav-kimi-oauth-token-freshness, cma-pav-kimi-multi-model-oauth-records, cma-pav-kimi-k3-moonshot-schema-proxy) and Check 5 in provider_aliases_challenge.sh (detector discovery, precedence, freshness, schema proxy, live kimi-alias freshness) — 15/15 PASS live.

Fixed

- **claude-providers verify <id> was unusable for OAuth providers** — it never injected the OAuth token, so verify-by-id always degraded to a false unverified. It now applies the same live-cred-file-first freshness order.
- **Layer-1 probes false-FAIL reasoning models** — 128-token budget was consumed entirely by chain-of-thought (k3, deepseek-v4-pro), yielding “empty content / sentinel missing” failures on working models. Probe budget is now 512 tokens.
- **Detector jq ARG_MAX overflow** — the full models.dev catalog was passed as a --argjson argument; only the kimi-for-coding models subtree is passed now (the bug silently yielded zero OAuth records and let an API-key record shadow the subscription).
- **Wrapper self-heal migration did not cover the new wrapper features** — the cma_run_provider migration markers stopped at v1.14.0, so every host upgrading kept a stale wrapper that (a) never started kimi_proxy for kimi-* (k3 400’d on every tool call — live-confirmed) and (b) never refreshed the OAuth token (401 after ~15 min). The _family_id and kimi-code/credentials/kimi-code.json markers now trigger regeneration.
- **Live legs blind to OAuth + overcounted account states** — verify_aliases_live.sh silently skipped OAuth aliases (“no key”), had no CLI-refresh fallback (stale snapshots → 400/401), lacked the family proxy discovery (kimi \$ref tests 400’d), used 32/64-token budgets that false-FAIL reasoning models (poe’s claude-sonnet-4.6 needs 512 to reach the tool call — proven live), and FAILED on account limits (weekly cap, fair-use 1313) instead of classifying them. alias_e2e_test.py had the same OAuth-key and staleness gaps. Both legs now: resolve the OAuth token through the full live-cred → CLI-refresh → snapshot chain, discover family proxies, use 256/512 budgets, and classify **SKIP-QUOTA** / **SKIP-TRANSIENT** / **SKIP-GATED** (aliases the gate already filtered) honestly instead of as passes or failures.

Testing

- New hermetic suite scripts/tests/test_kimi.sh (**33 assertions**): detector multi-record emission (live discovery + catalog union + offline fallback), expired-token CLI refresh,

OAuth-first precedence + no duplicates, API-key fallback mapping, launch-time token freshness (all three sources), cmd_verify OAuth injection, kimi_proxy schema normalization (5 cases), family proxy discovery markers. Full suite: 22/22 files ALL GREEN.

- Live (Kimi aliases FIRST, per release gate): all 4 aliases verified by layer-1 probes, layer-3 semantic (sentinel + independent judge), and real Claude Code launches through the aliases.

v1.14.0 — 2026-07-17 — Anti-bluff provider verification (strict filtering)

Fixed

- **Provider aliases passed verification while being broken at runtime.** Aliases such as huggingface were marked verified yet any real prompt (“Do you see my codebase?”) returned API failures (402 depleted credits, suspended accounts, unsupported models). A live sweep showed **12 of 19 aliases failing while status.json called them verified. Root causes (all fixed):**

1. *Layer-1 existence check was a bare GET /v1/models.* HTTP 200 proved only that the key was accepted — never inference, the selected model, or tool calling. **Fix:** scripts/providers-verify.sh now runs two live probes against the provider’s chat endpoint with the exact alias model: a sentinel probe (response MUST contain VERIFY_OK; 200-without-sentinel or error-in-200 is a bluff ⇒ failed) and a tool-calling probe (the model MUST emit a real tool call — Claude Code is tool-driven). Definitive rejections (401/402/403/404) ⇒ failed; only transient conditions (429/5xx/timeout/no-network) ⇒ unverified. Anthropic-native endpoints are probed in their native /v1/messages shape.
2. *Multi path (sync --multi) verified chat-only models.* model_verify.py never asserted the VERIFY_OK sentinel it requested, set verified=True unconditionally, and counted tool calling as zero required points (MIN_SCORE=25 == existence weight alone). **Fix:** sentinel is asserted (missing ⇒ anti-bluff failure), verified now **requires** a passed tool-calling probe, and the verification cache carries _cache_version so results from the old logic are never replayed.
3. *Billing/auth failures were classed as transient.* The semantic code-visibility layer exited 3 (“infra — honest SKIP, never downgrade”) on HTTP 402/401, so a credits-dead provider kept its stale verified. **Fix (LLMsVerifier submodule):** semantic-code-visibility now maps

- 401/402/403/404 on model-under-test calls to exit 1 (genuine negative — demotes), keeps 429/5xx/timeout as exit 3, and keeps judge-call failures always at exit 3 (a broken judge never demotes the model under test).
4. *The live alias verifier ignored tool calling.*
 verify_aliases_live.sh recorded “no tool call” but never failed on it — aliases passed with tool-less models. **Fix:** test 6 now uses an instructed tool call and is verdict-relevant.
 5. *The runtime-shaped e2e test was orphaned.*
 alias_e2e_test.py (tools, \$ref, cache_control, streaming through the real endpoint) was never invoked by anything. **Fix:** wired into run-proof.sh as leg 44 with an honest SKIP (exit 3) when no providers/network are present.
 6. *Probe URLs were mis-normalized for real provider bases.*
 Anthropic-native bases had their /anthropic prefix stripped (DeepSeek/Xiaomi 404’d on .../v1/messages when the served path is .../anthropic/v1/messages), and already-versioned bases (.../paas/v4 on Z.AI/BigModel coding plans) got a bogus /v1 inserted. **Fix:** the prefix is kept for native probes, versioned bases get only /chat/completions appended — in both providers-verify.sh and LLMsVerifier’s semantic-code-visibility (chatCompletionsURL rule, 11 new Go tests).
 7. *Single-attempt probes flapped on transient conditions.*
 Load-balanced gateways return occasional 400/404/412/000, and weaker models non-deterministically miss the sentinel or skip a tool call — working providers (kilo, siliconflow, sarvam) flipped to failed between syncs. **Fix:** exactly one retry for flappy codes and for flaky sentinel/tool-call outcomes; auth/billing codes (401/402/403) and consistent bluffs are never retried.
 8. *Layer-4 superpowers-TUI marker was model-phrasing-dependent.* Genuinely engaged sessions whose model summarized the framework in its own words (instead of the exact superpowers:<name> announce string) were failed. **Fix:** the marker also accepts vocabulary that can only exist when the skill content loaded (systematic-debugging, brainstorming, the invoke-before-response rule) — echo/refusal bluffs still cannot pass — and any transcript with "is_error":true is a hard FAIL (real API errors can’t masquerade as engagement).
 9. *Poe aliases failed real Claude Code launches through the proxy.* poe_proxy.py injected parameters only when missing/null, but Poe actually requires the properties key — Claude Code’s zero-argument tools ({"type":"object"} with no properties) were rejected with the misleading 400 Invalid 'tools': Field required (root cause reproduced and verified live against api.poe.com). **Fix:** fix_tools guarantees parameters.properties (and type:object) on every tool.
 10. *The live legs produced false FAILs on account/provider states and native transports.* verify_aliases_live.sh sent OpenAI-shaped Bearer requests to /anthropic bases (deepseek/xiaomi hung or 400’d) and had no retries, so

single 000/429 wobbles failed working aliases;
alias_e2e_test.py gave reasoning models (deepseek-v4-pro)
a 128-token budget that reliably came back empty, and
double-appended /chat/completions on versioned bases. **Fix:**
both legs speak native Anthropic shape under the kept /
anthropic prefix, versioned bases get exactly one /chat/
completions, reasoning models get a 512-token budget with
one empty-answer retry, and both legs now classify
honestly: **SKIP-QUOTA** (account funds: 402/
insufficient_quota/...) and **SKIP-TRANSIENT** (provider
capacity/timeouts/5xx) are reported separately and never
counted as passes or toolkit failures — the same FUNDS
distinction verify_claude_live.sh already made. Genuine
FAIL (auth, schema, bluff, no tool call after retry) stays
FAIL.

11. *A live API key leaked into committed proof evidence.*
opcode debug config embeds MCP server env
("TAVILY_API_KEY": "tvly-dev-..."), and cma_redact_secrets
only matched keys literally named apiKey|api_key|password|
secret|token — env-style names and the tvly-/nvapi-
prefixes slipped through into proof/10-debug-config.json.
Fix: the redactor now matches any JSON key NAME
containing key/token/secret/password/api-key and covers
the extra prefixes (\${VAR} placeholders still preserved);
extended hermetic tests in test_coverage.sh; full re-scan of
proof/ is clean.

Added

- **Tier C constitution/conformance verifier** scripts/tests/
verify_constitution.sh (design spec §7.4, implemented at
scripts/tests/ alongside the other live verifiers): CONST-051
submodule decoupling, §11.4.157 four-file doc lockstep,
§11.4.113 no force-push, §11.4.156 CI/CD disabled, §11.4.151
release-tag prefix, toolkit-owned fixture/rubric independence.
Wired into run-proof.sh as leg 45 — the proof suite is now six
legs: sandbox tier A, OpenCode live, providers live, aliases live,
alias e2e, constitution tier C.
- Hermetic test coverage: test_verify_scripts.sh grew from ~30
to 69 assertions (sentinel/tools/402/429/bluff/Anthropic-shape/
cache-version); new test_constitution.sh; test_providers.sh
loopback servers now answer the real chat+tools probe
shapes.
- **Challenges submodule:** new live anti-bluff challenge
challenges/scripts/provider_aliases_challenge.sh (static
pipeline-honesty checks + status.json freshness/consistency,
SKIP-OK when the toolkit is absent) and HelixQA-compatible
bank banks/examples/provider-alias-verification.json (6 test
cases), both registered in the docs/test-coverage.md ledger
(describe-runner 25/25).

Changed

- **Live filtering results (2026-07-17, all layers + real Claude Code launches):** every installed alias was re-verified with the strict pipeline and the ones that could not pass were removed (config dirs backed up as ~/.claude-prov-
<id>.preunify.*; re-adding is just claude-providers sync once the account is fixed). Removed: huggingface (+2,3), openrouter5, github-models, fireworks-ai, inference, novita-ai, sarvam, tencent-tokenhub, upstage, orphans zai/zhpuai, stale chutes4, xiaomi2/xiaomi3, rate-limited/unusable zai-coding-plan and zhpuai-coding-plan (429 fair-use limited / all-models-404 right now — honest removal, self-heals on the next sync), plus runtime-broken kimi-for-coding2 (k3 rejects Claude Code's \$ref tool schemas) and poe3 (gemma-4-31b aborts streams), and openrouter4 (nvidia/nemotron-nano-12b-v2-vl returns empty/malformed responses). **Final state: 28 aliases installed, every one verified through the full strict pipeline** (statuses, env files, and alias lines exactly consistent). overrides.json pins were corrected to models verified live today (openrouter, nvidia added; xiaomi fast model fixed to a served one).
- **LLMsVerifier submodule — strict scoring, no more bluff headroom:** removed the hard VerificationScore = max(score, 0.7) floor (every strict gate was vacuous); CodeVisibility confidence threshold 0.3 → 0.5 with rebalanced weights (a bare “Yes, I can see it” now scores 0.4 < 0.5); keyword matching uses \b word boundaries (“no” no longer matches “know”/“not”); round-1 sentinel check gained a prompt-echo guard (≥60-char verbatim fixture slice in the reply = genuine fail); the dead hardcoded HuggingFace endpoint api-inference.huggingface.co was replaced with https://router.huggingface.co/v1 in all production paths. Full Go suite: 59 packages green.
- docs/Provider_Aliases_User_Guide.md updated to the new verification semantics (§3 command table, §5 multi-alias verification, §7 rewritten).

v1.13.3 — 2026-07-17 — Session-sharing pipefail fix

Fixed

- **All aliases created separate, unshared sessions for the same project.** Switching between claude1, claude2, deepseek, xiaomi, etc. in the same project directory started a fresh conversation instead of resuming the shared one — no memory, context, or history continuity across aliases. **Root cause:** claude-session.sh has set -o pipefail. When

`cma_latest_session_id()` scanned for existing sessions with `ls -t ... | grep | head -1`, head closed stdin after one line, sending SIGPIPE to grep. With pipefail, the pipeline exited 141, and `set -e` aborted the script BEFORE it could return the session UUID. Every launch was treated as a first run, creating a new random-UUID session. **Fix:** Added `|| true` guard on the `head -1` pipeline in `cma_latest_session_id()` (`scripts/claude-session.sh` line 111). The guard catches SIGPIPE without affecting the captured output — `head` already printed the first line before exiting, so `latest` gets the correct UUID. All aliases now resolve to the same `--resume <uuid>`. **Verified with 200-file stress test** that exercises the exact pipefail condition.

Added

- 200-file stress test in `test_session.sh` that proves `claude-session flags` returns `--resume` (not `--session-id`) even with many session files, and that `latest-id` returns the most recent session.
- Investigation document at `docs/investigations/2026-07-17-session-sharing-root-cause.md` with full root cause analysis, evidence, and architecture notes.

v1.13.0 — 2026-07-10 — Project-scoped cwd-hook (session-resumption fix)

Fixed

- **Sessions were resumed for the wrong project when switching Claude Code aliases.** When the global `~/.local/bin/claude-cwd-hook` symlink pointed to one project's multitrack resolver (e.g. `atmosphere's`), every `claudeN` alias launch was redirected into that project's worktree BEFORE `claude-session` resolved the session — so the session was keyed to the `atmosphere` track, not the project the user was actually working in (e.g. `helix_ota`). Switching from `claude4` to `claude1` resumed an `atmosphere` Track-4 session instead of the `helix_ota` session. **Root cause:** `CMA_CWD_HOOK` was a single global singleton with no per-project awareness; the hook fired unconditionally and the toolkit had no mechanism for a repo to supply its own resolver. **Fix:** `cma_run` now resolves the `cwd-hook` in a three-tier order:
 1. `CMA_CWD_HOOK` env var (explicit override, unchanged),
 2. `<git-toplevel>/.claude-cwd-hook` (per-project hook — each repo gets its own multitrack resolver; prints nothing → stay in `$PWD`),
 3. `~/.local/bin/claude-cwd-hook` (global fallback, backward-compatible). A repo that needs its own track layout drops a `.claude-cwd-hook` at its git root; a repo that doesn't simply

omits it and keeps the global hook. The migration self-heals outdated `cma_run` wrappers via a new `_cma_hook_root` marker (detected on next `install.sh` / shell start).

Added

- Regression tests in `test_lib.sh` and `test_wrapper_exec.sh` prove the emitted `cma_run` body carries the project-scoped hook resolution code (`_cma_hook_root` marker, `git rev-parse --show-toplevel`, `.claude-cwd-hook` check) and respects the `CMA_CWD_HOOK` override + global fallback.

v1.12.3 — 2026-07-05 — Session-name sanitization (kebab-case)

Changed

- **Auto-derived session names are now sanitized to kebab-case.** `claude-session.sh` derives the session name from the project directory's basename. It now: lowercases the name; trims leading/trailing whitespace; collapses internal whitespace and underscores to `-`; strips any remaining characters that are not `[a-z0-9-]`; and collapses consecutive `-` before trimming leading/trailing `-`. This makes session names safe for the CLI and filesystem while remaining human-readable.

Added

- New regression tests in `test_session.sh` cover leading/trailing whitespace, multiple consecutive spaces, and stripping of special invalid characters.

v1.12.2 — 2026-07-05 — Native alias auto-registration + account-detection hardening

Fixed

- **Native `claude<N>` aliases were never created for pre-existing account dirs.** Running `install.sh` or `claude-unify.sh` only merged shared state; if account directories already existed (e.g. `~/.claude-milos85vasic`), no shell aliases were registered, so `claude1/claude2/etc.` were undefined. `claude-`

unify.sh now auto-registers a `claude<N>` alias for every detected account that lacks one.

- **Smart alias numbering:** an account dir literally named `~/.claude-claude4` keeps the `claude4` alias; remaining dirs fill the lowest free `claude<N>` slot instead of skipping over reserved numbers.
- **Bogus account detection:** `~/.claude-code-router` and `~/.claude-*.lock` directories are no longer treated as Claude accounts, preventing them from stealing alias slots or being merged into shared state.

Added

- Regression tests in `test_unify.sh` and `test_install.sh` prove that `install/unify` register native aliases for existing account dirs and preserve `claude<N>` basenames while filling gaps.

v1.12.1 — 2026-07-05 — Judge independence + resolve/robustness hardening

Addresses the v1.12.0 final whole-branch review's deferred items and the deep-research findings on LLM-as-judge bias.

Changed

- **Round-2 judge default is now a DIFFERENT model family.** `providers/judge.env.template` defaults to Groq / Llama-3.1 (`llama-3.1-8b-instant`) instead of DeepSeek. 2024-2026 research (arXiv:2508.06709 and others) shows a judge systematically favors its own model family — including validating that family's *wrong* answers, the exact failure layer-3 exists to catch — so defaulting the judge to a common subject (DeepSeek) was the worst case. Verified working live as an independent judge.
- **The semantic command distinguishes transport/infra failures from genuine verdicts.** The `LLMsVerifier semantic-code-visibility` command now exits **3** when a round-1/round-2 API call cannot complete (non-2xx, timeout, empty, connection error), vs exit **1** for a real negative verdict. `providers-semantic.sh` maps exit 3 → skip, so a transient judge/model hiccup is an honest SKIP and never downgrades the model-under-test (final-review I-1).

Added

- **Independence warning:** providers-semantic.sh warns (never fails) when the judge endpoint equals the model-under-test endpoint (same provider = same family = not independent).
- **xAI is now resolvable:** providers/overrides.json gains xai → <https://api.x.ai/v1> (the catalog lists xAI with no API base, so it previously resolved unmapped). Endpoint confirmed live.
- CONST-069 capability-boundary mandate in the LLMsVerifier submodule constitution (records the CONST-051 project-agnostic boundary under a non-colliding id).

Fixed

- A directory passed as the keys file (CMA_KEYS_FILE / --keys-file) now dies with a clear message instead of silently yielding “0 key vars”.

v1.12.0 — 2026-07-05 — Semantic code-visibility (layer 3) + live TUI verification (layer 4)

Adds two new provider-verification layers on top of the Phase-1 existence/tool-call checks, driven by the LLMsVerifier submodule's standalone, stdlib-only semantic-code-visibility Go command. The whole pipeline was proven end-to-end against real providers (redacted, real network): a genuinely code-seeing model (mistral-medium-2604) verifies (round-1 sentinel + round-2 judge 2/3); models that bluff or fail round-2 (chutes GLM-5.2-TEE — empty/timeout) are unverified; billing blocks (deepseek 402) are unverified — never a faked pass.

Added

- **Layer 3 — semantic code-visibility.** scripts/providers-semantic.sh renders the toolkit-owned rubric (providers/rubric/code-visibility-rubric.json) into a judge prompt and drives the Go command through a build-and-cache driver (scripts/claude-semantic-visibility.sh). Two rounds: a unique sentinel embedded in a code fixture (does the model actually see the code?), then an independent LLM-as-judge score ($\geq$ threshold). One-word contract verified|unverified|skip (exit 0/1/2). Wired into cmd_sync; keys move via env only (never argv). CONST-051 boundary held — the submodule stays project-agnostic, receiving fixture/prompt/ judge-prompt/sentinel only as CLI args.
- **Layer 4 — live superpowers-TUI verification.** scripts/verify_superpowers_tui.sh launches real Claude Code through a

provider alias and confirms the superpowers plugin engages with no trust/overwrite prompt — the only thing that flips a provider to fully verified. Honest SKIP when preconditions (real claude/key/network) are absent; the engagement classifier is hardened against false-PASS on prompt-echo, and a live negative-case test (neutral prompt → marker must NOT match) proved it does not false-verify.

- **claude-providers verify <id> [--deep]** — single-provider deep re-verify.
- **Tier-B live verifier** (scripts/tests/verify_providers_live.sh) runs layers 3–4 per installed provider, writes secret-redacted evidence + an aggregate proof/providers-summary.json; already wired into run-proof.sh.

Fixed

- **--refresh-aliases was not byte-idempotent** — `cma_provider_write_alias` re-ran the full `cma_ensure_alias_file` self-heal migrations on *every* alias line, which could non-deterministically reposition the `cma_run_provider` function and produce a different file on a second refresh (the session hook runs this on every shell). Now only bootstraps the alias file when absent. (Root-caused from a captured diff; 42 flake-loops + repeated full-suite runs now deterministic.)
- Three bugs found by real execution of the new layer-3 path: `claude-semantic-visibility.sh --help` failed pre-build; `providers-semantic.sh` discarded the driver's JSON evidence to `/dev/null`; the judge base URL was not normalized (a trailing `/v1` doubled to `/v1/v1/chat/completions`).
- Final-review hardening: `cmd_sync/cmd_sync_multi` heal a stale/outdated `cma_run_provider` wrapper once per sync (the byte-idempotence change had removed the per-shell self-heal, so a pre-Phase-2 gate-less wrapper could still launch); the layer-4 engagement classifier now requires the namespaced `superpowers:<name>` form (a bare skill word could false-verify); and `verify --deep` treats a layer-4 verifier crash as an honest SKIP rather than a clean verify.

Notes

- The default `providers/judge.env.template` judge is DeepSeek. For strongest results, configure `providers/judge.env` with a judge from a **different model family** than the provider under test — 2024-2026 research (e.g. arXiv:2508.06709) shows same-family judges systematically favor their own family's outputs, including validating wrong answers. A different-family default, an `xAI overrides.json` base-url entry, and the submodule boundary-contract constitution id are tracked for a follow-up.

v1.11.2 — 2026-07-03 — Token-limit guard + re-enabled provider proxies + Poe tool cap

Ran every provider alias through REAL Claude Code executing /using-superpowers (the user's reproducing prompt), in CLI mode with scrubbed env, and root-caused every genuine failure. Three real toolkit bugs fixed (below); all remaining non-passes are account-side (insufficient funds, rejected/absent keys, or a key with no chat entitlement) — not toolkit defects.

Fixed

- **Input token-limit 400** — scripts/lib.sh (cma_run_provider) now exports
CLAUDE_CODE_AUTO_COMPACT_WINDOW="\$CMA_PROVIDER_CONTEXT_LIMIT"
before the transport branch, so Claude Code knows each provider's real input window and auto-compacts (at window - 13000) before a request overshoots it. Fixes the user-reported 400 ... exceeded model token limit: 262144 (requested: 311786) on kimi-for-coding and any provider whose window is smaller than the ~1M Claude Code assumes for Anthropic's endpoint. Fully dynamic/parametrized: the value is the per-model limit.context from the models.dev catalog, persisted in each provider .env. Guarded by [[-n ...]]; applies to **both** transports. (CLAUDE_CODE_MAX_OUTPUT_TOKENS only ever capped **output** — it could not fix an **input** overflow.) Verified live: kimi's /using-superpowers now compacts and succeeds.
- **All provider proxies were silently disabled** — cma_run_provider resolved compatibility proxies via \$LIB_DIR/proxy/..., but LIB_DIR is a repo-only variable that does not exist in the self-contained alias file, so it expanded empty and **no proxy ever started** (Poe, etc.). Now resolves against the installed location \${SHARED_DIR:-\$HOME/.claude-shared}/proxy (where install.sh copies scripts/proxy/*.py). This is why Poe returned 400 Invalid 'tools': Field required — its request never went through the proxy that injects the required parameters field.
- **Poe tool-count limit** — Poe rejects requests carrying more than ~216 tool definitions with the same misleading 400 Invalid 'tools': Field required (verified count-based: 215 accepted, 220 rejected, independent of payload size). On this host Claude Code's large MCP-plugin load emits 400+ tools. scripts/proxy/poe_proxy.py now caps the tool list to POE_MAX_TOOLS (default 200), dropping only overflow mcp_... tools so **every** built-in Claude Code tool is preserved. Parametrized via the POE_MAX_TOOLS env var. Verified live: Poe's /using-superpowers now returns a successful result.

Changed

- **cma_ensure_alias_file migration** — added `CLAUDE_CODE_AUTO_COMPACT_WINDOW` and `_cma_proxy_dir` as regeneration markers, so an already-installed `cma_run_provider` predating either fix is transparently regenerated on the next alias-file touch.
- **scripts/tests/verify_claude_live.sh** — `reclassify_fail` now maps a direct 400 whose body says the model is unknown/invalid to **BADKEY** (an account that can list models but not invoke them — e.g. inference here), while a 400 with no model-rejection marker (a real launch-layer defect) stays FAIL and is never masked.
- Corrected misleading comments that claimed the output-token cap fixed the input token-limit error; documented the two guards as independent halves.

Added

- **scripts/tests/test_poe_proxy.sh** — hermetic tests for the Poe proxy: parameters injection, the tool-count cap, built-in-tool preservation, the `POE_MAX_TOOLS` override, and end-to-end `fix_request`.
- **scripts/tests/test_providers.sh** — hermetic regression tests: the emitted `cma_run_provider` exports the auto-compact window from `CMA_PROVIDER_CONTEXT_LIMIT` only when non-empty; the migration regenerates an outdated wrapper lacking the guard while preserving surrounding alias lines.
- **scripts/tests/proof/claude-live-superpowers-cli.txt** — evidence matrix: every provider alias launched through REAL Claude Code running `/using-superpowers`, classified PASS / FUNDS / BADKEY / NOKEY / FAIL.

v1.11.1 — 2026-07-03 — Live per-alias Claude Code verification (CLI + TUI) + provider fixes

Investigated the reported “most provider aliases fail with an API error.” Root cause was NOT a broad implementation defect: launched every alias through real Claude Code (scrubbed env) and probed each provider API directly. 9 aliases pass end-to-end; 2 had genuine model-config drift (fixed below); the remaining failures are all **account-side** (insufficient funds, invalid/expired keys, missing keys, or a plan with no chat model) — not toolkit bugs.

Added

- **scripts/tests/verify_claude_live.sh** — end-to-end live verification of every provider alias through REAL Claude Code in BOTH modes: **CLI** (-p ... --output-format json, authoritative) and **TUI** (driven under a PTY). Each launch runs in a scrubbed env; TUI runs from a throwaway temp cwd so it can never resume a real conversation (the cross-alias .claude.json sync would otherwise auto-resume one). Outcomes are classified **PASS / FUNDS / BADKEY / NOKEY / FAIL** so account problems are never miscounted as toolkit bugs; on a launch FAIL it probes the provider API directly to recover the true cause (e.g. a ccr hang on an upstream 401/429 → BADKEY/FUNDS, not FAIL).
- **scripts/tests/lib/pty_drive.py** — pexpect PTY driver for the interactive Ink TUI (boot, accept any trust prompt, type prompt, capture transcript, quit).
- **scripts/tests/lib/classify_live.py** — shared transcript classifier.

Fixed

- **huggingface** — strong/fast pinned to non-reasoning coder models (Qwen/Qwen3-Coder-480B-A35B-Instruct / Qwen/Qwen3-Coder-30B-A3B-Instruct). The previous models emitted their answer as hidden reasoning_content, returning empty content at low max_tokens and reading as broken. **Verified green** in CLI + TUI.
- **kilo** — strong/fast pinned to verified free-tier models (nvidia/nemotron-3-super-120b-a12b:free / nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free). The old x-ai/grok-build-0.1 is a **paid** model this key can't access (401 → >100s stall) and the old fast baidu/cobuddy:free is retired (404). **Verified green** in CLI + TUI.
- **inference** — base URL corrected https://inference.net/v1 → https://api.inference.net/v1 (old host 301-redirects). NOTE: this key's plan currently exposes no general chat model, so the alias still errors 400 until a chat-capable plan/key is supplied — documented, not silently “fixed.”
- **novita-ai** — defensive swap off the retired fast model sao10K/L3-8B-stheno-v3.2 (404).
- **claude-providers.sh (present_key_vars)** — skip declared-but-empty key vars so an empty key (e.g. SARVAM_API_KEY=) no longer spawns a broken alias that only errors at launch.

Verified (live on host)

- **9 aliases PASS** end-to-end in CLI (and TUI smoke): chutes, huggingface, kilo, kimi-for-coding, nvidia, opencode, poe, siliconflow, zai-coding-plan.

- Non-PASS are **account-side, not toolkit bugs**: FUNDS — deepseek, fireworks-ai, novita-ai, openrouter, upstage, xiaomi, zhipuai; BADKEY — github-models, tencent-tokenhub; NOKEY — sarvam; plan-limited — inference.
- Sandbox providers suite **113/113**; shellcheck clean; python files compile.

v1.10.8 — 2026-07-01 — noclobber-safe router-provider config write

Fixed

- **Every router-transport provider alias broke under set -o noclobber.** `cma_run_provider` runs in the user's interactive shell; when that shell has `noclobber` set, the router-config rewrite `jq ... "$cfg" > "$tmp"` failed with *"cannot overwrite existing file"* (the just-created `mktemp` target already exists), silently dropping the update so `claude-code-router` launched with a stale/empty config → API errors. Switched the write to the force-clobber operator `>|`. (`lib.sh`)
- **Existing installs self-heal:** added a regeneration trigger so `cma_ensure_alias_file` rewrites an outdated `cma_run_provider` that lacks the `>|` fix (plain function-body changes previously did not re-trigger on install).

Added (tests)

- `test_providers.sh`: regression asserting the emitted `cma_run_provider` uses `>|` (not a bare `>`) for the router-config write, plus a functional proof that `>` is blocked by `noclobber` while `>|` succeeds.

Verified

- Suite **18/18 green**; shellcheck 0. Root cause reproduced (`set -C; echo > "$(mktemp)"` → "cannot overwrite"), fix deployed + confirmed on the host (deployed alias write line = `>| "$tmp"`).

Note (not a toolkit bug)

- A 46-alias provider sweep shows native providers reach their APIs and return **account-side** errors (e.g. 402 Insufficient Balance) — provider billing/key state, not a toolkit fault. **## v1.10.7 — 2026-06-29 — Shared-items drift guard + audit closure**

Added (tests)

- **test_unify.sh** — a drift guard asserting `claude-unify.sh`'s `SHARED_ITEMS` equals `lib.sh`'s `CMA_SHARED_ITEMS` (used by `claude-add-account`) minus the intentional `CLAUDE.md` special-case (`unify` promotes it via `sync_claude_md`). Catches the documented hazard of adding a shared item to one list only.
- Cleaned a pre-existing SC2319 lint in `test_unify.sh`.

Audited — no code changes (3 parallel investigators; every finding independently checked)

- **Unify + rollback engine: clean** — enabled `Plugins union`, `history.jsonl dedup`, `settings.json merge` (malformed-input safe), `plugin-manifest path rewrite`, and `rollback` all verified correct across 170+ assertions + edge cases.
- **OpenCode sync (opencode_sync.py): clean** — idempotent + additive verified; two reported “bugs” were refuted against the code: the `setdefault` “can’t update existing keys” IS the documented never-clobber design, and the `--enable-all secret nit` is not a security issue (an unresolved secret can’t leak) and matches “enable everything” semantics.
- **Runtime sync + add-account: clean** — the `SHARED_ITEMS` “drift” is the intentional `CLAUDE.md` special-case (now locked by the guard above); `sync-state` private-key isolation, corrupt-input handling, and `add-account` idempotency verified.

Verified

- Suite **18/18 green**; shellcheck 0.

v1.10.6 — 2026-06-29 — Committed credential-leak regression test

Added (tests)

- **test_lib.sh** — a committed security regression for `cma_merge_claude_json`: two accounts with distinct `userID/``oauthAccount` and disjoint projects are merged; asserts each account keeps its OWN private auth keys (no cross-account leak in either direction) and the projects subtree is unioned both ways. The function previously had only indirect coverage (via the full `unify` workflow); this locks the property an audit verified by hand this session.

Verified

- Suite **18/18 green**; shellcheck 0.

v1.10.5 — 2026-06-29 — Provider ‘null’ field normalization + coverage

Fixed

- **A missing JSON field could write CMA_PROVIDER_MODEL='null' (and TRANSPORT, alias name) into a provider env file**, launching the provider with a bogus model.
cma_provider_write_env normalized base/fast/context/max “null”→empty but missed model and transport; claude-providers multi-sync also extracted strong_model/transport/alias_name with bare jq -r (no // empty, unlike the already-correct context_limit/max_output). Fixed at both the source (// empty on every extraction) and the choke point (normalize model+transport in cma_provider_write_env). Reproduced (='null'), confirmed fixed (='').

Added (tests)

- **test_providers.sh** — regression asserting a null model/transport/base/etc. is normalized to empty; no field ever contains the literal 'null'.
- **test_session.sh** — EXECUTION tests for the hint subcommand (run on every bare launch, previously only string-matched) and for cma_project_root’s git-toplevel + symlink (pwd -P) branches.

Audited — no change needed (independently verified, not taken on trust)

- cma_merge_claude_json: NO cross-account credential leak; projects unioned; corrupt input skipped gracefully (verified with crafted 2-account inputs).
- BSD/macOS portability: no unguarded GNU-isms (the 3 readlink -f hits are comments; stat -f/-c branch + cma_realpath guards present).
- jq robustness: the @tsv sync paths render null as empty (safe); two reported 2>&1 “error-leak” findings were FALSE — there is no 2>&1 on those lines.

Verified

- Suite **18/18 green**; shellcheck 0.

v1.10.4 — 2026-06-29 — set -e/pipefail abort fixes + hardened test coverage

Fixed

- **claude-providers list / remove aborted on a provider with no alias line.** Under set -euo pipefail, the alias-name probe `grep ... | sed | head -1` returns 1 (no match) when a provider's .env exists but its alias line is absent (manual edit / partial setup); pipefail propagated the failure and set -e killed the subshell (list) or the function before `rm -f` (remove). Guarded both with `|| alias=""`. (claude-providers.sh)
- **cma_ensure_alias_file aborted on an alias file lacking export CLAUDE_BIN=.** The CLAUDE_BIN-migration probe `grep -m1 '^export CLAUDE_BIN=' ...` returned 1 on an older/hand-edited alias file and aborted the function mid-run under set -e. Guarded with `|| _cur_cb=""`. (lib.sh)

Changed (tests)

- **test_providers.sh** — replaced the AT-RISK fixed-window `grep -A40 '^cma_run()'` assertions (the push marker had drifted to within 9 lines of the window edge, the same brittleness that already broke -A30 once) with full-body awk extraction; added EXECUTION regressions that run the real `claude-providers list/remove` against an alias-less provider and assert no abort.
- **test_coverage.sh** — added a regression that EXECUTES `cma_ensure_alias_file` against an alias file with no `export CLAUDE_BIN=` line and asserts it completes.
- **test_session.sh** — added EXECUTION tests for the hint subcommand (run on every bare launch, previously only string-matched): exits 0, writes only to stderr, names the `snake_case` project, handles an empty label.

Verified

- Suite **18/18 green**; shellcheck 0. All three aborts reproduced (RED) and confirmed fixed (GREEN); the providers fix proven RED on a guard-stripped copy. Found via 3 parallel investigator subagents, each finding independently reproduced before fixing.

v1.10.3 — 2026-06-29 — Execution-level wrapper test coverage

Added

- **test_wrapper_exec.sh** — the first hermetic test that actually *executes* the generated `cma_run` wrapper (every other suite only string-matches its emitted body, so a runtime bug — a set `-e` abort, a dropped unset, wrong call order — could ship past a green suite). It drives `cma_run` with a stub `CLAUDE_BIN` env-recorder plus stub `claude-session/claude-sync-state`, then asserts `RUNTIME` guarantees: provider-env isolation (a leaked `ANTHROPIC_BASE_URL/AUTH_TOKEN/MODEL` is genuinely cleared *before* claude runs), session flags reach claude on a bare launch, `sync-state` pull fires before launch and push after, explicit args pass through verbatim with no session-flag injection, plus a non-vacuity guard proving the stub claude really executed. Proven **RED** on a dropped unset, **GREEN** on the real wrapper.

Verified

- Suite **18/18 green**; shellcheck 0.

v1.10.2 — 2026-06-29 — Self-healing rc source lines + strict rc tests

Fixed

- **Dangling source `".../aliases.sh"` lines in rc files.** A transient or moved alias-file path could leave a source line in `~/.bashrc/~/.zshrc` pointing at a deleted file, so every new login shell printed `-bash: .../aliases.sh: No such file or directory`. `cma_ensure_alias_file` now **prunes** any rc source/. line whose `aliases.sh` target no longer exists (self-heal on the next install), and recognizes an existing source line across `./source` and `$HOME/~`/absolute forms, so re-installs never accumulate duplicate source lines.

Added

- **test_rc_sourcing.sh** (10 strict assertions) — reproduces the bug class the hermetic suite missed (it sandboxes `$HOME` and never inspected or *sourced* the rc files): prune drops dangling / keeps valid + comments + unrelated lines, ensure self-heals, **a fresh shell sources the rc with NO error** (the reported symptom), idempotent (exactly one source line after 3 calls), and cross-

form dedup. Proven RED on the old behavior, GREEN on the fix.

Verified

- Suite **17/17 green**; shellcheck 0.

v1.10.1 — 2026-06-29 — Robust cma_run wrapper assertions

Fixed

- **test_claude.sh used a fixed grep -A30 window** to scan the cma_run body and silently missed the sync-state push marker once the body grew with the v1.10.0 apply-color calls (push slipped past line 30) — failing the suite against a v1.10.0-installed alias file even though the wrapper itself was correct. It now extracts the full function body (awk header → closing brace), robust to future growth.

Verified

- Suite **16/16 green** against the v1.10.0 wrapper; shellcheck 0.

v1.10.0 — 2026-06-29 — Auto-applied per-alias session color + coverage/wiring

The per-alias session color is now **auto-applied** (it was only a hint in v1.9.x), plus self-healing for a stale CLAUDE_BIN and several closed test-coverage gaps.

Added

- **Auto-applied per-alias session color.** Each bare alias launch now writes the alias's color into the session as an agent-color record — the exact record Claude Code's /color writes — via the new claude-session apply-color, called by cma_run/cma_run_provider (before launch to colour a resumed session; after exit to colour a freshly-created one). Deterministic $\text{md5}(\text{label}) \bmod 8$ over red/blue/green/yellow/purple/orange/pink/cyan: each alias gets a stable, distinct colour, and switching the same session between aliases re-colours it. Verified **LIVE** on claude 2.1.195 — written, idempotent, persists across --resume. (Prompt-bar rendering must be

confirmed visually: `/color` is TUI-only and `claude -p '/color x'` is a no-op, so record injection is the only non-interactive mechanism. See [docs/SESSION_COLOR.md](#).)

- Test coverage: `test_install.sh` (executes `install.sh` in a sandbox — symlinks, alias file, idempotency), `test_verify_scripts.sh` (`model_verify.py`
 - `providers-verify.sh`), `test_session` apply-color tests (incl. the `set -e` regression), `test_coverage` B7 (CLAUDE_BIN resolver), B8 (CLAUDE_BIN migration), B9 (apply-color wired into both wrappers). `run-proof.sh` now also runs the previously-orphaned `verify_aliases_live.sh`.

Fixed

- **Stale CLAUDE_BIN self-heals.** Existing installs whose alias file pointed CLAUDE_BIN at a non-existent path (e.g. `~/.local/bin/claude` where `npm` put `claude` in `~/.npm-global/bin` — the `amber.local` case) now rewrite it to a resolved, executable `claude` on the next `install/ensure`.
- A `set -e/pipefail` bug where `apply-color` aborted before writing on a session that had no existing `agent-color` record.

Verified

- Suite **16/16 ALL GREEN; shellcheck 0**. Color injection proven **LIVE** on real `claude 2.1.195` (`write / idempotent / persist-across-- resume / recolour-on-alias-switch`).

v1.9.2 — 2026-06-29 — Hermetic CLAUDE_BIN resolver test

Fixed

- `test_coverage.sh` **B7 “fallback when nowhere” was not hermetic** and failed on hosts that have a real `/usr/local/bin/claude` (caught live on `thinker.local`): the resolver *correctly* returns the system `claude` there, but the test wrongly assumed “`claude nowhere`” was achievable under a sandboxed `HOME/PATH` (it can’t mask absolute system paths). Dropped that one assertion; the load-bearing discovery cases (explicit `CLAUDE_BIN`, `~/.npm-global/bin` discovery) stay covered. Runtime behavior unchanged.

Verified

- Suite **14/14 green on all five hosts** (this host, `mistborn`, `thinker`, `amber`, `nezha`); `shellcheck 0`.

v1.9.1 — 2026-06-29 — CLAUDE_BIN resolves across per-host install locations

A patch found during the live multi-host rollout of v1.9.0.

Fixed

- **Alias launches failed where claude was installed outside ~/.local/bin.** npm i -g @anthropic-ai/claude-code lands in different prefixes per host (~/.npm-global/bin, Homebrew, ~/.local/bin); the toolkit's fixed CLAUDE_BIN default mis-pointed on those hosts, so every claudeN/provider launch failed "No such file or directory" (amber.local needed a manual symlink to work). cma_resolve_claude_bin now prefers an explicit CLAUDE_BIN, then \$PATH, then the known locations (~/.local/bin, ~/.npm-global/bin, /opt/homebrew/bin, /usr/local/bin), with a ~/.local/bin fallback.

Verified

- Suite **14/14 green; shellcheck 0**. test_coverage.sh B7 covers explicit / npm-global / fallback resolution. v1.9.0's auto-session naming confirmed installed + **live-validated** (create-named + legacy-rename) on all five hosts: this host, mistborn, thinker, amber, nezha.

v1.9.0 — 2026-06-29 — Per-project auto-sessions that actually work live + zero-coverage tests

A minor release that makes the v1.8.0 "auto session-per-project" feature do what it promised. As shipped, opening any alias gave an **unnamed** session; three root causes — all reproduced and fixed against the real claude 2.1.195 binary, then proven **LIVE end-to-end** — are corrected here, plus test coverage for two zero-coverage utilities and a documentation refresh.

Fixed

- **Per-project auto-session naming never actually named the session — now proven LIVE.** Three independent root causes:
 - **Legacy/unnamed sessions were never renamed.** The launcher only --resume'd an existing session and never passed --name, so a session created by an older wrapper or

by plain `claude` stayed unnamed forever. Fix: always pass `--name` on resume too. Proven live — `claude --resume <id> --name <x>` renames a previously-unnamed session (custom-title `<NONE>` → `<x>`), contradicting the docs but confirmed empirically.

- **The session-existence check used a run-collapsing slug.** It collapsed runs of non-alnum to one - (`s/[^A-Za-z0-9]+/-/g`), but `claude` slugs **per char**, so paths with consecutive non-alnum segments (hidden dirs, `/tmp/.private`, `__pycache__`) false-negated the lookup and **re-created** instead of resuming. Fix: per-char slug (`s/[^A-Za-z0-9]/-/g`), matching the real on-disk dir names.
- **`cma_run` self-heal regenerated only on a missing unset `ANTHROPIC_` marker.** Wrappers predating auto-session carried that marker but not the `claude-session` one, so they never regained the integration. Fix: regenerate when **either** marker is missing.

Added

- **`docs/SESSION_COLOR.md`** — resolves the previously dangling reference, documenting per-project auto-session naming and the honest `/color` limitation: in `claude 2.1.195`, `/color` is **TUI-only** (no CLI flag, no `settings.json` key, no env var — verified against the binary and the docs), so the toolkit can only print a deterministic per-alias hint, never auto-apply it.
- **`test_toon.sh` (9 assertions) and `test_bootstrap.sh` (39 assertions)** — both utilities previously had **zero** coverage. `toon` (hermetic, SKIP-if-no-node): `toon.mjs` encode/decode round-trip, the `toon_encode.py` python→mjs chain, and non-zero exit on invalid JSON. `bootstrap` (hermetic): `claude-bootstrap --count 2 --yes` in a sandbox `$HOME` asserting account dirs, shared symlinks, private-file isolation, alias lines, and the documented refuse-to-clobber re-run behavior.
- **`test_coverage.sh` B6** asserts the emitted `cma_run / cma_run_provider` bodies actually carry the auto-session integration (bare-launch guard, `claude-session` flags, `eval set -- apply`, color hint) — the session script's own unit tests can't see the wrapper — plus a **self-heal regression**: a stale `cma_run` missing the `claude-session` marker is regenerated (exactly one `cma_run()`, provider-env isolation retained, aliases preserved). `test_session.sh` updated for `--name-on-resume` and a per-char-slug regression (a `/.cfg/` path must resume, not re-create).
- **Docs refresh.** README rewritten as a project landing page; `scripts/README` refreshed with the full current script inventory; the long-form guide gained “Per-project auto-session & per-alias color” and “TOON utility” sections; the `/color` notes pinned to the verified `claude 2.1.195` (superseding older 2.1.178 references).

Verified

- Full suite **14/14 ALL GREEN; shellcheck 0**. The session fix proven **LIVE end-to-end** on the real `claude 2.1.195` binary — fresh create, resume, and legacy-unnamed rename all confirmed. New tests are non-vacuous (concrete expected values / negative controls). Installed live on this host and validated.

v1.8.1 — 2026-06-29 — Merge-engine correctness + portability hardening

A patch release: an adversarial correctness audit of `claude-unify`'s merge engine plus a BSD/GNU portability pass over the test + proof tooling. All fixes/hardening — **no new features**.

Housekeeping: a divergent mirror lineage that re-created `v1.7.11` (`1e975e5`) was merged back into main resolved to **OURS** (local already carries `v1.7.11` → `v1.8.0` and later fixes that supersede it), leaving a tree byte-identical to HEAD so all four mirrors converge on one lineage; the containers submodule was fast-forwarded to latest main (`71d3256` → `67ed35a`).

Fixed

- **history.jsonl merge fused records across a source missing its trailing newline.** `merge_history_jsonl` cat'd sources into a temp first, gluing one file's last line onto the next file's first line → two entries collapsed into one invalid-JSON line. Fix: feed files straight to `awk` (fresh record per file). Regression **R1** (RED before, GREEN after).
- **enabledPlugins union dropped "any true".** The `jq` used `+/` (rightmost-wins), so a plugin enabled in an earlier account but false in the lexically-last account ended up disabled for everyone — contradicting the documented "any true survives" guarantee. Fix: `OR-of-true` reduce over every account. Regression **R2**.
- **A single malformed settings.json aborted the whole unify — and naive guarding then risked silent config loss.** The multi-file `jq -s` ran unguarded under `set -e` (`settings` is item 15 of 16), halting mid-run. Merely skipping the merge was worse: `link_to_shared` still replaced each valid account's real `settings.json` with a symlink to a never-written target (a dangling link → silent loss, exit 0). Final fix (hardened after adversarial review): validate each file with `jq empty` and merge only the valid ones (a malformed sibling is excluded, not fatal), and `link_to_shared` refuses to create a link when the shared target is absent. Regression **R3** (asserts the valid account's settings stay readable, not just that unify exits 0).

- **Directory-merge conflicts were resolved by lexical account name, not recency.** `merge_dir_into_shared`'s second pass overlaid only `ACCOUNTS[-1]` (alphabetically-last) while claiming to bias toward the “most recently active” account, so a stale account sorting last could clobber fresher memory/*.md. Fix: overlay every account with `rsync -au` so the newest-mtime file wins each conflict, independent of name/order. Regression **R4**.
- **Rollback left dangling symlinks.** Unify symlinks every shared item into each account; for an item an account never had there is no `.preunify` backup, so rollback's restore loop never visited it and the symlink dangled once `SHARED_DIR` moved aside. Fix: after restoring backups, remove any leftover symlink whose target points into `SHARED_DIR` (skipping the shared store itself). Regression **R5**.
- **Rollback restored a non-deterministic backup.** `find -print0` was unsorted, so when a path had several `.preunify.*` backups an arbitrary one was restored. Fix: `sort -z` (timestamps are `YYYYMMDDHHMMSS` = lexical-chronological) so the oldest — the true pre-unify original — wins. Regression **R6**.
- **test_unify.sh B2 was a vacuous PASS.** It called `cma_realpath` (a `lib.sh` function) without sourcing `lib.sh`, so the call errored to empty and the assertion compared `"" == ""` — the symlink target was never verified. Fix: `source lib.sh + set +e` (matching every sibling test that uses `lib` functions directly). Now prints the real resolved `SHARED_DIR/plugins/cache` path.
- **Portability: 3 GNU-only constructs broke the test/proof tooling on macOS** (the shipped runtime toolkit was already clean). `readlink -f` (no `-f` on BSD) in `assert_symlink_to/` `test_unify.sh` returned empty → spurious symlink pass/fail, fixed with a self-contained `_assert_realpath` in `assert.sh + cma_realpath` in `test_unify.sh`; `sed -E 's/\x1b.../'` (\xNN is GNU-sed-only) in `run-proof.sh/verify_opencode_live.sh` left ANSI in, skewing `grep -c` counts, fixed by building the ESC byte via `printf '\033'`; unguarded timeout (GNU coreutils) in `verify_opencode_live.sh`, fixed by resolving `timeout/gtimeout` once and degrading if absent.
- **De-vendored node_modules.** `node_modules/@toon-format/toon` was committed by an accidental “Auto-commit” yet load-bearing (`scripts/toon.mjs` imports the bare specifier; `toon_encode.py` shells out to it) with nothing ever running `npm install`. Removed from the tree (`git rm --cached + gitignore / node_modules/`). Proven by fresh-clone simulation: `ERR_MODULE_NOT_FOUND` before, encodes correctly after.
- **mktemp portability.** Standardized every bare `mktemp [-d]` to the templated `mktemp [-d] "${TMPDIR:-/tmp}/cma.XXXXXX"` form `CLAUDE.md` prescribes (BSD `mktemp` requires a template; only GNU tolerates a bare call) across `lib.sh`, `claude-unify/` `providers/opencode-sync/session/install`, and the test harness (the sandbox keeps its `cma-test.` prefix for the cleanup safety check).

Added

- **B5 token-limit guard coverage** (test_coverage.sh). The v1.8.0 context_limit/max_output path (cma_provider_write_env → CMA_PROVIDER_CONTEXT_LIMIT/CMA_PROVIDER_MAX_OUTPUT → cma_run_provider exporting CLAUDE_CODE_MAX_OUTPUT_TOKENS) shipped with **zero** tests — the only v1.8.0 fix lacking one. 4 cases / 6 concrete-value assertions: round-trip (262144/32768), null→empty normalization, 7-arg back-compat, and the emitted wrapper carrying the export.
- **npm install step in install.sh** (soft — warns, never hard-fails, when npm is absent; core unify/add-account needs no Node), so a fresh clone gets @toon-format/toon without a vendored tree. curl-install.sh inherits it via delegation.
- **+16 regression assertions** — 6 in test_coverage.sh (B5) and 10 in test_unify.sh (R1-R6 above), each written RED-before / GREEN-after.
- **Documented two deliberate merge/sync trade-offs** in-code so they are explicit rather than silent: cma_merge_claude_json replaces (not element-unions) array values; claude-sync-state pull/push is last-writer-wins (no portable mutex is worth its stale-lock failure modes for a per-launch hook).

Verified

- Full suite **12/12 ALL GREEN; shellcheck 0**; all .py compile under python3 -W error. Each bugfix proven **RED-before / GREEN-after**; the de-vendor proven via fresh-clone simulation (node scripts/toon.mjs + toon_encode.py); ESC-strip verified functionally; the post-merge tree confirmed byte-identical to HEAD. Installed live on this host and validated against all existing aliases (3 native + 44 provider) + claude-list-accounts.

v1.8.0 — 2026-06-29 — Alias isolation + token-limit guard + per-project auto-sessions

A systematic-debugging pass fixing three reported issues plus a new session-per-project feature. Every root cause was reproduced and the fix proven with physical evidence before shipping.

Fixed

- **CRITICAL — aliases cross-contaminated API endpoints across sessions.** cma_run_provider exports ANTHROPIC_BASE_URL/ AUTH_TOKEN/MODEL/ SMALL_FAST_MODEL into the interactive shell, and native cma_run did **not** clear them — so running a provider alias

(e.g. xiaomi) and then a native alias (claude1) in the same shell made the native one inherit the provider's endpoint (api.xiaomimimo.com). cma_run now unsets those four vars before launch. Proven live: after a leaked xiaomi env, native launch shows ANTHROPIC_BASE_URL=<unset>. Existing installs auto-regenerate the wrapper (migration keyed on the new unset ANTHROPIC_marker).

- **Token-limit 400 (“exceeded model token limit: 262144”).** The models.dev catalog's per-model limit.context / limit.output were read for ranking but never emitted, so Claude Code overshoot a provider's real context window. providers_resolve.py now emits context_limit + max_output; providers_generate.py and cma_provider_write_env write CMA_PROVIDER_CONTEXT_LIMIT / CMA_PROVIDER_MAX_OUTPUT into each .env; and cma_run_provider exports CLAUDE_CODE_MAX_OUTPUT_TOKENS from it. Proven: kimi-for-coding now resolves context_limit=262144 max_output=32768 from the live catalog.
- **“workspace has not been trusted” warning on launch.** Confirmed NOT a merge bug (trust propagates across accounts correctly); the warned project was simply never trusted under any account. The launch wrapper now writes projects[<root>].hasTrustDialogAccepted=true for the launching project via the new claude-session helper.

Added

- **Auto session-per-project (claude-session.sh).** Every bare alias launch (native or provider) now resumes — or, the first time, creates — one long-lived Claude session per project root: stable --session-id (UUID derived from the git-root path), --name set to the root dir basename in lowercase snake_case (Android 15 → android_15). Explicit args/flags are always respected verbatim. Verified against the real claude CLI: --session-id creates, --resume resumes.
- **Per-alias color hint.** A deterministic alias→color mapping over Claude Code's real palette (red blue green yellow purple orange pink cyan); printed as a /color <x> tip on launch. (Investigated thoroughly: /color is a TUI-only command with no CLI flag / settings key / writable persistence, so it cannot be auto-applied — the toolkit suggests it rather than faking it.)
- **test_session.sh** — 27 hermetic assertions for name/id/color/flags/trust/ git-root behavior. **run-all.sh is now 12 files / 60 assertions, ALL GREEN.**

Verified

- Full suite **12/12 ALL GREEN; shellcheck 0**; all .py compile under python3 -W error. All four items proven end-to-end against the live catalog and the emitted alias file.

v1.7.12 — 2026-06-28 — One-line curl installer

Added

- **curl-install.sh** — one-line bootstrap installer:

```
curl -fsSL https://raw.githubusercontent.com/vasic-digital/claude-toolkit/main/scripts/curl-install.sh | bash
```

Detects platform (Linux/macOS) and shell, auto-installs missing hard dependencies (jq, rsync, awk) via the system package manager (apt/dnf/apk/pacman/brew), clones (or pulls if already present) the repo with all submodules recursively to ~/claude-toolkit, runs install.sh, and prints next-steps. Idempotent; re-runnable. Install dir overridable via CLAUDE_TOOLKIT_DIR env var.

- **README.md** — curl one-liner added at the top of the Install section.
- **test_curl_install.sh** — 22 hermetic tests covering syntax, permissions, URL correctness, submodule cloning, idempotency, platform detection, dependency checks, error handling, and next-steps output.

Verified

- bash -n + shellcheck 0 on curl-install.sh and test_curl_install.sh.
- run-all.sh **11/11 ALL GREEN** (was 10; +test_curl_install.sh).

v1.7.11 — 2026-06-28 — Round-4: coverage-gap regression tests, toon recursion guard, arg validation

Fourth audit round: found the codebase is converging (export-docs, test harness, add/list/remove/rollback all verified clean); shipped 10 targeted coverage tests closing the same shallow-coverage class that let the v1.7.10 enable-plugins bug ship; plus 2 LOW code findings.

Fixed

- **toon_encode.py fallback_encode**: unbounded recursion on deeply nested JSON. A crafted input with >1000 levels would blow

Python's default stack and exit with an unhandled traceback instead of encoding. Added a `_depth` guard; beyond 64 levels emits compact JSON as a safe fallback.

- **toon.mjs encode-file / decode-file**: running `toon.mjs encode-file` with no argument produced a confusing `TypeError` from Node's `fs.readFileSync`. Now prints
Error: encode-file requires a filename argument + exit 1.

Added — coverage-gap regression tests (the same class that let enable-plugins)

ship) - **B9 (HIGH) — cma_ensure_alias_file migration path** (`test_coverage.sh`): builds a realistic old `cma_run_provider()` body lacking `claude-sync-state`, calls `cma_ensure_alias_file`, asserts the body is migrated, the following alias `claude1=` survives, and `cma_run_provider()` appears exactly once. - **B3 (HIGH) — _cma_q bash quoting in cma_provider_write_env** (`test_coverage.sh`): sources a `.env` with a model name containing a literal single quote and asserts it round-trips intact; also asserts an injection payload does NOT execute on source (mirrors the already-tested Python `q()`). - **B1 (HIGH) — absorb_default_plugins** (`test_unify.sh`): creates a real plugin file under `$HOME/.claude/plugins/cache/` before unify; asserts it lands in `$SHARED_DIR/plugins/cache/`. - **B2 (HIGH) — link_default_plugin_subdirs** (`test_unify.sh`): asserts `$DEFAULT_DIR/plugins/cache` becomes a symlink into `$SHARED_DIR/plugins/cache` after unify, and that re-running unify doesn't create a second backup. - **B4 (MEDIUM-HIGH) — sync_claude_md seed branches** (`test_unify.sh`): branch (b) seeds `$DEFAULT_DIR/CLAUDE.md` and asserts it wins; branch (c) removes it and gives an account a `CLAUDE.md`, asserts that one wins.

Verified

- `run-all.sh` **10/10 ALL GREEN** (coverage now $39+10=49$ assertions; unify now $43+7=50$); **shellcheck 0**; all `.py` compile under `python3 -W error`; `node --check toon.mjs` clean; `toon_encode 500-level-nest` no longer crashes; `toon.mjs missing-arg` gives clean error + exit 1.

v1.7.10 — 2026-06-28 — Round-3 audit: enable-plugins bug fix, path-traversal guards, proxy robustness

Third audit round (deep dive on the less-covered surface: `opencode_sync`, `claude-unify` merge, the `poe` proxy, bootstrap). Fixes verified centrally.

Fixed

- **cma_enable_plugins silently enabled NO plugins when given 3 or more** (lib.sh). The jq --arg index was derived as $\$ \{ \# \text{args} [@] \} / 2$, but each iteration appends **three** elements — so for the default 4 always-on plugins it produced arg names p0,p1,p3,p4 while the jq program referenced \$p0..\$p3; \$p2 was undefined, jq failed, 2>/dev/null swallowed the error, and enabledPlugins was left empty. Replaced the derived index with a dedicated counter. Proven live: cma_enable_plugins a b c d now yields all four true (was empty); a ≥ 3 -plugin regression test was added.

Security (defense-in-depth)

- **Path traversal via unvalidated provider id** in claude-providers.sh cmd_show / cmd_remove: \$id was interpolated into <dir>/\$id.env and then cat/rm -f'd without validation. Now rejected unless it matches [A-Za-z0-9._-] (blocks ../), matching cma_provider_write_alias.
- **opcode_sync.py \${CLAUDE_PLUGIN_ROOT} path traversal:** a malicious installed plugin could set an arg like \${CLAUDE_PLUGIN_ROOT}/../../../../tmp/evil.js and --enable-all-local would have OpenCode exec the traversed path. Expansion now lexically contains the result to the plugin dir; an escaping value is left unexpanded (fails safe).
- **cma_write_alias now rejects whitespace in the config dir** (lib.sh): an unquoted space silently word-split the alias into a bogus command — now a clear error instead of a broken alias.

Robustness

- **poe_proxy.py:** resolve_refs gained a recursion-depth guard (a circular \$ref previously crashed the request handler with RecursionError); the success-path gzip.decompress is now guarded like the error path, so a corrupt gzip body no longer propagates.

Verified

- run-all.sh **10/10 ALL GREEN** (coverage now 32 assertions incl. the enable-plugins + injection regressions); **shellcheck 0**; all .py compile under python3 -W error.
- cma_enable_plugins fix proven live (4 plugins → all true); opcode containment + id validation proven with PoCs. The model-verification / alias- write path is unchanged from v1.7.9's live-proven 137 models / 32 aliases.

Audit (round 3) — verified clean

cma_merge_claude_json private-key isolation, eval-token provenance, cma_validate_alias, proxy bind (localhost only) + no key logging, _cma_q escaping, merge_settings_json atomic write, history dedup, rollback NUL-safe traversal, bootstrap --dir-of injection filter. (opencode_sync --enable-all intentionally bypasses the needs-secret guard — operator opt-in, documented.)

v1.7.9 — 2026-06-28 — Hardening round 2: injection-safe alias writes, broadened secret redaction, docs accuracy, shellcheck 0

A second multi-agent audit + hardening pass on top of v1.7.8 (adversarial security audit + doc-accuracy audit + lint sweep, fixes verified centrally).

Security

- **Provider id / config dir can no longer inject shell via the alias file** (lib.sh cma_provider_write_alias / cma_write_alias). Both interpolate values into alias name="..." lines that the shell **re-parses on invocation**, and jq @tsv does not escape ". They now reject shell metacharacters (provider id restricted to [A-Za-z0-9._-]; config dir rejects " \$ \ ; & | < > () and newline). Proven: afoo"; touch ...` payload is rejected, no command runs, the hostile alias is never written.
- **Keys-file read no longer breakable by a quote in the path** (claude-providers.sh cmd_sync_multi). The old bash -c "set -a; source '\$keysf'; ..." let a single quote in the keys-file path break out of the string. Replaced with an isolated subshell (set +e; set -a +u; . "\$keysf"; set +a; eval ...) — the same safe pattern cmd_sync already used. Proven with a do n't/ path.
- **.env value quoting** (providers_generate.py): q() now POSIX single-quote-escapes embedded quotes (mirrors lib.sh _cma_q), so a catalog value containing a quote can't inject when cma_run_provider sources the .env. Proven: an injection payload is neutralized to a literal.
- **xtrace secret leak** (lib.sh cma_run_provider): the indirect key read is now wrapped in set +x/restore so an active set -x in the user's shell can't echo the key to the terminal or a redirected log.
- **Broadened secret redaction + guard**: cma_redact_secrets() (verify_opencode_live.sh) and the committed-proof scan guard (test_lib.sh) now also catch sk-ant-, hf_, AIza, xoxb-/xoxp-/xoxs-, pc-, re_, secret_, and JWTs — regardless of JSON field name — closing the gap where arbitrary MCP env-var names

(e.g. NOTION_API_KEY) slipped through the original six-name allowlist.

Fixed

- **install.sh** used **readlink -f** (absent on BSD/macOS) for its symlink up-to-date check — missed by the v1.7.7 sweep. Now uses `cma_realpath`; the `test_lib.sh` guard scans `install.sh` too.
- **verify_aliases_live.sh** **hardcoded one developer's account dirs**, producing false FAILs on every other host. Now discovers accounts dynamically and skips dirs that don't exist.
- Dead code / cruft: `providers_generate.py` (unused import, dead vars, `lambda→def`, a no-op `provider_id + ('' if ... else '')`); `model_verify.py` (unused import `hashlib`); `model_verify.py` docstring `--key → CMA_PROBE_KEY`.

Docs

- Long-form doc + READMEs + CLAUDE.md corrected against the code: macOS rc-file caveat (`~/ .zshrc` only), the test table now lists all 10 suites, the full installed-command list (`+claude-providers/claude-sync-state/ claude-bootstrap`), repo-relative paths (was `~/Documents/scripts/`), a new `claude-bootstrap` section, the `CMA_PROBE_KEY` security model in §11, and a refreshed date stamp.

Quality

- **shellcheck: 93 → 0** across all scripts. Added `.shellcheckrc` (`external-sources=true`) which resolves the sourced-file warnings properly; fixed the `$?-after-condition` (SC2319) test idioms, SC2015/SC1090/SC1003; the one remaining reserved no-op flag carries a justified inline disable.

Verified

- `scripts/tests/run-all.sh` **10/10 ALL GREEN**; **shellcheck 0**; every `.py` compiles under `python3 -W error`.
- Injection PoCs (provider id, config dir, `.env` value, keys-file path) all proven neutralized; broadened redaction proven against `AIza/hf_/JWT/etc`.
- Live sync `--multi`: **137 models verified, 32 aliases** across 8 providers (opencode 4, poe 33, chutes 7, huggingface 6, nvidia 30, openrouter 14, siliconflow 38, xiaomi 5), zero `CMA_PROBE_KEY/unbound errors` — identical to the v1.7.8 baseline, so the new key-read path is non-regressive end-to-end.
- 4-host byte-parity + 10/10 suite re-verified after deploy.

v1.7.8 — 2026-06-28 — Secret hygiene (argv + committed-proof leaks), dead-code fix, coverage tests

Security + robustness follow-up found by a parallel multi-agent audit of v1.7.7. Four independent subagents fixed disjoint file sets; integration + the full suite + live multi-model verification were run centrally.

Security

- **API key no longer passed on argv** (model_verify.py + claude-providers.sh). cmd_sync_multi invoked model_verify.py --key "\$token", placing the secret verbatim in /proc/<pid>/cmdline and ps aux output — readable by any user on a multi-user host. The key now flows via the CMA_PROBE_KEY environment variable (set per-command, not exported); model_verify.py reads it from the environment and errors clearly if unset. The --key flag is removed entirely.
- **API key no longer passed to curl on argv** (verify_aliases_live.sh). Six live-probe calls used -H "Authorization: Bearer \$key". The header is now written to a mktemp'd, chmod 600 config file consumed via curl --config (portable on GNU + BSD curl) and removed via an EXIT/INT/TERM trap.
- **Leaked secrets purged from committed proof artifacts** (committed in 24bc379, rolled into this release): the OpenCode live verifier wrote resolved opencode debug config / mcp list output — which contained a real provider key and a DB connection-string password — verbatim into the committed proof dir. The three artifacts are redacted; the generator (verify_opencode_live.sh) now redacts via cma_redact_secrets() before writing (raw dump → .raw temp → redacted file → .raw removed). **Operator follow-up still required:** rotate the leaked key and decide on a git-history scrub — the values remain in history on all four remotes.

Fixed

- **Unreachable code** in verify_aliases_live.sh: exit \$failed sat *before* the Claude-alias test function and its caller, making them dead (shellcheck SC2317). exit \$failed moved to the final statement.
- **Fragile \$? capture** in test_list.sh: grep ...; [[\$? -ne 0]] then assert_eq 0 \$? read \$? from the wrong command. Now captures rc=\$? immediately.
- **Unquoted glob** in claude-sync-state.sh:67: "\$HOME"/\${ACCOUNT_PREFIX}prov-* / → "\$HOME/\${ACCOUNT_PREFIX}"prov-* / so only the intended * globs.

- **SyntaxWarning: invalid escape sequence '\ ' in** providers_resolve.py: the usage docstring's \ line-continuations are now a raw string (r""").

Added

- **test_coverage.sh** — 11 new hermetic tests (19 assertions) covering previously-untested lib.sh behavior: cma_ensure_alias_file (fresh / idempotent / old-format migration preserving unrelated lines), cma_can_prompt (CMA_NONINTERACTIVE=1 and no-tty both non-interactive), cma_enable_plugins (JSON shape + additive + the jq // falsy-vs-null upgrade), cma_link_shared_items (every CMA_SHARED_ITEMS entry becomes a symlink into \$SHARED_DIR, idempotent), and stats-cache.json newest-by-mtime selection.
- **Proof regression guard** (test_lib.sh): scans scripts/tests/proof for provider-key prefixes and URL user:password@ creds, counting suspect lines so a failure never re-echoes a secret.

Verified

- scripts/tests/run-all.sh — **10/10 ALL GREEN** locally (was 9; +test_coverage.sh).
- Live multi-model verification (claude-providers.sh sync --multi, real HTTP probes with the host's real keys): **137 models verified, 32 aliases generated** across 8 providers (opencode 4, poe 33, chutes 7, huggingface 6, nvidia 30, openrouter 14, siliconflow 38, xiaomi 5). Zero CMA_PROBE_KEY-unset and zero unbound variable errors — the env-var key path works end-to-end. (Providers with 0 verified are external: dead/paid keys, HTTP 401/402/403, WAF blocks — not toolkit regressions.)
- The new proof secret-scan guard immediately earned its keep: on first cross-host run it flagged a **stale, pre-redaction proof dir on all three remote hosts** (3 files with literal secrets), which were then re-synced with the redacted artifacts.
- model_verify.py / providers_resolve.py compile clean under python3 -W error.

v1.7.7 — 2026-06-28 — Portable realpath (BSD portability hardening), set -u edge fix, regression tests

Follow-up hardening release found by a parallel multi-agent audit of v1.7.6.

Fixed

- **readlink -f** → **portable cma_realpath** at three sites: `claude-unify.sh` (already_linked_to_shared and merge_settings_json) and `claude-list-accounts.sh` (the link check). `readlink -f` is absent on older macOS and on other BSDs (FreeBSD/NetBSD); there the checks silently fail — making `claude-unify` re-link every shared item on each re-run (accumulating stale `.preunify.*` backups) and `claude-list-accounts` report linked accounts as “not linked”. **Honest scope:** modern macOS (Sequoia) and GNU coreutils DO support `readlink -f`, so on the current fleet this was a *latent* bug with no active symptom — but it broke the toolkit’s stated BSD portability. Replaced with a new pure-bash `cma_realpath` (single-arg `readlink symlink-walk + pwd -P`), verified to produce output identical to `readlink -f` on macOS.
- **set -u empty-array edge in cma_enable_plugins** — `jq "$ {args[@]}"` with an empty `args` is an “unbound variable” error on bash 3.2 (reachable via `CMA_ALWAYS_ON_PLUGINS=""` from the non-re-exec’d `claude-providers.sh`). Guarded with `${args[@]+"${args[@]}"}`.

Added

- **cma_realpath** portable canonicalizer in `lib.sh`.
- **Regression tests** (`test_lib.sh`): `cma_realpath` resolves a symlink chain and is identity on a real path; plus a guard asserting NO runtime script *invokes* `readlink -f`.

Verified

- `scripts/tests/run-all.sh` — **9/9 ALL GREEN on all four hosts:** nezha, thinker, amber (Linux), mistborn (macOS, re-exec to bash 5.3, BSD userland).
- `cma_realpath` output confirmed byte-identical to `readlink -f` on macOS.

Audit findings (v1.7.6 — no code change required)

- Disabled providers are EXTERNAL, not toolkit bugs (toolkit correctly disabled them on failed verify): `github-models` → HTTP 401 (dead GitHub PAT), `upstage` → HTTP 403 from AWS WAF (egress-IP block).
- `api_keys.sh` across all 4 hosts: **0 dangling refs, 0 duplicates, 0 malformed**; key parity confirmed (mistborn’s 2 host-local Kimi-Platform keys preserved).
- Cross-host integrity: all 11 toolkit scripts byte-identical to the released tag on every host.
- Known/deferred: published tags `v1.2.0` (gitlab) and `v1.5.0` (gitlab/gitverse/gitflic) point to older commits than local —

reconciling needs a force tag push; left for a maintainer decision.

v1.7.6 — 2026-06-28 — Always-non-interactive execution, alias-file integrity, macOS/bash-3.2 portability, 4-host rollout

Fixed

- **Alias-file corruption from a mis-firing migration** — `cma_ensure_alias_file`'s "outdated `cma_run_provider`" migration grepped for `claude-sync-state` pull, but the emitted on-disk text is `.../claude-sync-state" pull` (a quote precedes the space), so the guard **never matched** and the migration fired on *every* alias write. Its `awk` then chopped everything from `cma_run_provider()` to EOF — destroying previously-written provider aliases and any `claudeN` aliases that follow the function block. This silently corrupted the alias file on multi-provider / multi-account hosts. Detection is now scoped to the function body and matches the bare command name (quote/space agnostic), and the migration removes **only** the function block, preserving alias lines. This was the single root cause of the failures across `test_providers.sh`, `test_claude.sh`, and `test_add_remove.sh`.
- **set -u abort while sourcing the keys file** — provider sync sourced `~/api_keys.sh` inside a `set -euo pipefail` subshell. A dangling reference in the user's keys file (e.g. `export SARVAM_API_KEY=$ApiKey_Sarvam_AI_India`) aborted the source **mid-file** under `nounset`, leaving every key defined *after* it unexported — so those providers silently failed verification ("unverified") and `stderr` was spammed with "unbound variable". Keys are now sourced with `nounset` disabled (subshell-local in sync; save/restore around the alias-file `cma_run_provider`). Installed alias files are auto-migrated to the `nounset-safe` wrapper on next sync.
- **macOS / bash-3.2 portability of the test harness** — `tests/run-all.sh` used `mapfile` (bash 4+), so the **entire suite failed to run on stock macOS**. Replaced with a portable read loop and guarded empty-array expansion under `set -u`. Same fix applied to `test_lib.sh` and `tests/lib/sandbox.sh` (empty `${arr[@]}` expansions are unbound on bash 3.2). The suite now runs green on macOS bash 3.2.

Added

- **CMA_NONINTERACTIVE + automatic TTY detection** — a new `cma_can_prompt` helper makes every prompt (`claude-add-account`,

claude-remove-account, claude-bootstrap) fall back to its non-interactive default whenever no terminal is available (CI, SSH without a PTY, the test sandbox) or when CMA_NONINTERACTIVE=1 is exported. Toolkit execution is now **always non-interactive off a terminal**. Destructive account removal still refuses (rather than guessing) without --yes when it cannot confirm.

- **Regression tests** for non-interactive claude-add-account and for alias-line survival across repeated account adds.
- **test_export.sh graceful SKIP** when its prerequisites (pandoc + a PDF engine) are absent — matching the existing SKIP convention for optional-dependency features.

Multi-host rollout (nezha · mistborn.local · thinker.local · amber.local)

- Distributed ~/api_keys.sh to every host via a **no-loss merge** (host-local keys preserved — e.g. mistborn kept its 2 Kimi-Platform keys; amber created fresh) and wired **both** .bashrc and .zshrc to source it on every host.
- Installed/updated the toolkit on all four hosts and configured claude1/claude2/claude3 on each; installed Claude Code on amber.
- Ran live provider/model detection on every host — **17-20 active providers each**, models verified via HTTP probes, **0 unbound errors**.

Verified

- scripts/tests/run-all.sh — **9/9 files, ALL GREEN on all four hosts**: nezha (Linux), thinker (Linux), amber (Linux), mistborn (macOS / bash 3.2).
- Cross-host: both rc files source api_keys.sh; claude1/2/3 + poe/deepseek/xiaomi aliases present on every host.

v1.7.5 — 2026-06-28 — Cross-provider / resume session visibility fix

Fixed

- **Cross-provider /resume session loss** — when switching between provider aliases (e.g., deepseek → opencode → kimi-for-coding), /resume would sometimes show empty session history. Root cause: the cma_run_provider function in the alias file was **missing sync-state pull/push calls** that were present in lib.sh. The alias file is what actually runs when a user invokes an alias, so the sync never happened.
- **Migration for outdated alias files** — added automatic detection and regeneration of outdated cma_run_provider

functions in `lib.sh`. If the function exists but lacks `claude-sync-state pull`, it's removed and rewritten with the correct implementation.

- **Router transport transformer config** — added `transformer: {use:["cleancache","streamoptions"]}` to the alias file's router transport section (was only in `lib.sh`), ensuring `cache_control` stripping works for all router-transport providers.

Root Cause Analysis

The `cma_run_provider` function in `lib.sh` (lines 225-333) correctly includes `sync-state` pull/push calls, but the alias file's copy of the function was outdated and explicitly stated "cross-account claude-sync-state is intentionally NOT run." This meant: 1. Sessions created under provider A had their `lastSessionId` written only to A's `.claude.json` 2. When switching to provider B, B's `.claude.json` still had its own (different) `lastSessionId` 3. `/resume` read B's `lastSessionId` and couldn't find A's session

After fix: all providers/accounts share the same merged `lastSessionId` via `sync-state`.

Verified

- **Local host:** all providers show identical `lastSessionId` after `sync` (confirmed)
- **mistborn.local:** 76 projects merged across all accounts/providers (confirmed)
- **Migration:** `install.sh` correctly detects and fixes outdated alias files on both hosts

Tests

- Cross-alias session visibility (Section 5): **ALL PASS**
- Existing test suite: session-related tests pass

v1.7.4 — 2026-06-26 — Kimi provider fix + AWS IaC MCP disabled by default

Fixed

- **Kimi Code provider base URL** in `scripts/providers/overrides.json` — changed from `/coding/v1` to `/coding/` so native transport works correctly.
- **AWS IaC MCP timeout** — removed `aws-dev-toolkit/awsiac` from the default OpenCode MCP allowlist in `scripts/claude-`

opencode-sync.sh. The server consistently timed out on connection and is now configured but disabled by default.

Changed

- Regenerated Claude_Multi_Account_Fine_Tuning.{html,pdf,docx} from current markdown.
- Refreshed proof artifacts in scripts/tests/proof/.

Tests

- Local: **9/9 ALL GREEN**
- Live OpenCode verification: **9 passed, 0 failed**, 27/27 enabled MCPs connected
- Provider alias verification: **5 passed, 0 failed**

v1.6.6 — 2026-06-21 — TOON integration for token-efficient prompts

Added

- **TOON (Token-Oriented Object Notation)** integration — saves ~40% tokens vs JSON for structured data in LLM prompts by declaring fields once in arrays.
- **scripts/toon.mjs** — Node.js TOON utility (encode/decode/demo)
- **scripts/toon_encode.py** — Python wrapper for TOON encoding
- **docs/TOON_Integration.md** — comprehensive guide on using TOON with Claude Code
- **package.json** — @toon-format/toon v2.3.0 dependency

Token Savings

- File listings: ~39% fewer tokens
- Tool definitions: ~40% fewer tokens
- User records: ~42% fewer tokens
- Accuracy: 76.4% (vs JSON's 75.0%)

Note

TOON formats message CONTENT for token savings. API transport remains JSON (providers require it). HTTP/3 and compression require provider-side support.

Tests

- 8/8 ALL GREEN

v1.6.5 — 2026-06-21 — Poe proxy fix (alias file + install)

Fixed

- **Poe proxy not starting from alias** — proxy logic was only in `lib.sh`, not in the alias file's `cma_run_provider` function. The alias file is what actually runs when a user invokes an alias. Added proxy detection + auto-start to the alias file.
- **install.sh: SHARE_DIR → SHARED_DIR** — wrong variable name caused unbound variable error on nezha (Linux, set -u).
- **install.sh: auto-copy proxy scripts** to `~/.local/share/.../proxy/` during install.

Verified

- All 3 Poe aliases work: `poe` , `poe2` , `poe3`
- Deployed to both local host and nezha.local

Tests

- 8/8 ALL GREEN

v1.6.4 — 2026-06-21 — Poe proxy fix for tool compatibility

Fixed

- **Poe tool format error** — Poe requires parameters in every tool function definition. Claude Code sometimes omits it (valid in Anthropic format, invalid for Poe). Added `poe_proxy.py` that auto-fixes tools before forwarding to Poe API.
- **Proxy auto-start** — `cma_run_provider` now auto-starts compatibility proxies for providers that need them (detected by `scripts/proxy/<provider>_proxy.py`).

Verified

- All 3 Poe aliases work through proxy: `poe` , `poe2` , `poe3`

Tests

- 8/8 ALL GREEN

v1.6.3 — 2026-06-21 — Poe provider (382 models, 3 aliases)

Added

- **Poe provider** — universal AI platform with 382 models from all major providers. OpenAI-compatible API at <https://api.poe.com/v1>. Chat, code, image gen, video gen, TTS, STT, and more.
- **3 aliases**: poe (claude-sonnet-4.6 + gpt-5.4-mini), poe2 (gpt-5.5 + deepseek-v4-pro-e), poe3 (grok-4 + gemini-3.1-pro)
- **key-aliases**: POE_API_KEY + ApiKey_Poe → poe
- **Tool calling verified** on claude-sonnet-4.6, gpt-5.4-mini, deepseek-v4-pro-e, grok-4
- **382 models categorized**: 130 chat/reasoning, 16 code, 40 image gen, 17 video gen, 12 TTS, 1 STT, 166 other
- **Documentation**: full Poe section in Provider_Aliases_User_Guide.md

Verified

- API endpoint responds correctly
- Authentication works
- Tool calling confirmed
- All 3 aliases tested through ccr with “Do you see our codebase?” — all YES

v1.6.2 — 2026-06-21 — Chutes provider documentation + model update

Changed

- **Chutes provider models updated** — catalog was stale. Chutes now offers 13 TEE (Trusted Execution Environment) models. Updated strong=zai-org/GLM-5.2-TEE, fast=Qwen/Qwen3.6-27B-TEE.
- **Chutes documentation** added to Provider_Aliases_User_Guide.md with full model table, TEE explanation, pay-per-use note, and setup instructions.

Verified

- Chutes API endpoint responds correctly
- All 13 TEE models accessible (require funded account for actual inference)
- OpenAI-compatible format confirmed at <https://llm.chutes.ai/v1>

v1.6.1 — 2026-06-21 — cache_control fix + E2E tests

Fixed

- **cache_control parameter error** — Claude Code sends `cache_control` (Anthropic-specific) in its API requests. ccr forwarded this to OpenAI-compatible endpoints which reject it with HTTP 422. Fixed by adding ccr's built-in `cleancache` transformer to every provider config, which strips `cache_control` before forwarding to the provider.

Added

- **alias_e2e_test.py** — end-to-end alias verification script that tests each alias by sending requests through ccr and verifying responses work without errors.

Verified working (all aliases tested with “Do you see our codebase?”)

- `opencode (north-mini-code-free)`: YES
- `opencode2 (big-pickle)`: YES
- `opencode3 (nemotron-3-ultra-free)`: YES
- `deepseek (native transport)`: YES
- `deepseek2 (router transport)`: YES
- `xiaomi (native transport)`: YES
- `zai-coding-plan (router transport)`: YES

v1.6.0 — 2026-06-21 — Multi-alias provider system

Added

- **Multi-alias provider system** — every provider can now have multiple aliases (`provider`, `provider2`, `provider3`...) exposing ALL

working models, not just the top 2. Verified via live HTTP probes with anti-bluff detection.

- **model_verify.py** — comprehensive model verification & scoring engine. Tests every model for a provider via HTTP probes, scores on 7 dimensions (existence 25pts, tool_call 20pts, reasoning 15pts, context_window 15pts, streaming 10pts, latency 10pts, free_tier 5pts). Anti-bluff detection prevents false positives (HTTP 200 with error body, empty responses, boilerplate errors). 24h verification cache to avoid re-testing.
- **providers_generate.py** — multi-alias generation from verified models. Pairs models into alias groups of 2 (strong + fast), handles odd count (last model reused for both positions), single model (used for both positions). Generates env files, shell aliases, and overrides.json entries.
- **claude-providers.sh --multi** — new flag for sync that triggers the full verification + multi-alias generation pipeline. Additional flags: --max-aliases (default 5), --min-score (default 25), --verify-concurrency (default 5).
- **Endpoint normalization** — /anthropic endpoints auto-converted to /v1 for OpenAI-compatible probing during verification.
- **Submodules updated** to helix_translate-2.3.1: LLMsVerifier (ModelVerifier, Seed, xiaomi provider), challenges (anti-bluff §11.4, chaos/stress tests), containers (deploy-stack).

Changed

- Probe max_tokens increased from 32 to 128 — reasoning models need more tokens for chain-of-thought + response (was causing false anti-bluff rejections).
- User-Agent header added to HTTP probes (some APIs require it).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures.

Usage

```
# Standard sync (2 models per provider, as before)
claude-providers sync
```

```
# Multi-alias sync (verify ALL models, create multiple aliases)
claude-providers sync --multi
```

```
# With options
claude-providers sync --multi --max-aliases 10 --min-score 20
```

v1.5.1 — 2026-06-20 — Linux stat fix + nezha deployment

Fixed

- **stat -f %m on Linux** — the mtime cache check in `claude-providers.sh` used `stat -f %m || stat -c %Y` as an `||` chain. On Linux, `stat -f` succeeds (returning filesystem info, not mtime), so both outputs merged into garbage ("File: ...1781634386"), causing File: unbound variable under `set -u`. Fixed with case "\$(`uname -s`)" to pick the correct flag per platform.

Deployment

- **nezha.local** (Linux x86_64) deployed and verified: 19 providers activated, 100/100 provider tests pass, 5/5 live verifier pass, cross-alias sync confirmed. Evidence in `scripts/tests/proof/90-nezha-deployment.txt`.

Full test suite

- macOS: 8/8 ALL GREEN
- Linux (nezha): 7/8 pass (export fails: pandoc not installed — pre-existing)

v1.5.0 — 2026-06-20 — Cross-alias session visibility

Added

- **Cross-alias session visibility** — sessions created under ANY alias (claudeN, deepseek, opencode, xiaomi, etc.) are now visible from every other alias via `/resume`. Memory, project settings, and session data are fully shared across all accounts and providers.
- **claude-sync-state.sh extended** — now discovers provider dirs (`~/.claude-prov-*`) alongside account dirs for its `.claude.json` merge. Provider sessions participate in the same lightweight jq merge that keeps account sessions in sync.
- **cma_run_provider sync-state hooks** — the provider wrapper now calls `claude-sync-state pull` before launch and `claude-sync-state push` after exit, matching the `cma_run` pattern. Previously provider sessions were intentionally excluded from sync; now they participate fully.
- **Sandbox test coverage**: 10 new assertions proving cross-alias merge (sessions from account→provider,

provider→account, account→account all visible after sync).

Providers test 90 → 100 assertions.

- **Live verification:** lastSessionId for a real project confirmed identical across all dirs (3 accounts + 1 provider). 61 projects merged in every .claude.json. Evidence in scripts/tests/proof/80-cross-alias-sessions.txt.

Changed

- scripts/claude-sync-state.sh — provider dirs included in merge targets
- scripts/lib.sh — cma_run_provider wrapper updated with sync-state pull/push
- Alias file aliases.sh — updated cma_run_provider function (re-installed)

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 10 new assertions for cross-alias session visibility.

How it works

1. claude-sync-state pull merges every account's + provider's .claude.json into the launching dir before Claude Code starts (including lastSessionId, allowedTools, MCP config, etc.).
2. Claude Code launches with the merged state — /resume sees all sessions.
3. claude-sync-state push merges the post-session .claude.json back out after exit, so the next alias to launch picks up the new session.
4. The sessions/ directory was already shared via symlink — this release ensures .claude.json project settings are also merged.

Performance

- Adds ~1-2 seconds overhead per provider launch (jq merge of .claude.json across all dirs). Same overhead that claudeN aliases already have.

v1.4.0 — 2026-06-20 — OpenCode Zen provider alias

Added

- **opencode provider alias** — OpenCode Zen curated AI gateway with **21 free models** (all \$0 cost, all support tool calling +

reasoning) and 49 paid models. The alias uses **router transport** (ccr) targeting the OpenAI-compatible endpoint <https://opencode.ai/zen/v1/chat/completions>.

- **Model overrides:** strong = big-pickle (free stealth model, 200K context, reasoning + tool_call), fast = deepseek-v4-flash-free (free, 200K context, reasoning + tool_call). Pinning is deliberate — auto-selection would pick nemotron-3-ultra-free (1M ctx) as strong and trinity-large-preview-free (131K, no reasoning) as fast, both suboptimal for coding workloads.
- **key-aliases.json mappings:** ZEN_API_KEY → opencode and ApiKey_Opencode_Zen → opencode (both key vars present in the user's keys file).
- **overrides.json pin:** strong_model=big-pickle, fast_model=deepseek-v4-flash-free (no transport/base_url override needed — catalog values are correct).
- **Sandbox test coverage:** resolver tests (key-alias mapping for both key vars, router transport from @ai-sdk/openai-compatible npm, zen/v1 base_url from catalog, model override beats auto-selection, stale-model-never-selected guards) + sync e2e tests (env file, alias, config-dir + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 69 → 90 assertions.
- **Live endpoint verification:** GET /v1/models HTTP 200; POST /v1/chat/completions round trip HTTP 200 with correct text for big-pickle (stealth, cost=\$0, reasoning_content present) and deepseek-v4-flash-free (cost=\$0); additional free models (mimo-v2.5-free, nemotron-3-ultra-free, north-mini-code-free) all HTTP 200 with cost=\$0. Evidence in scripts/tests/proof/70-zen-live.txt (secret-free).
- **Docs:** dedicated opencode section in docs/ Provider_Aliases_User_Guide.md (full free models table, setup, usage, live-verified notes, stealth model explanation).

Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the opencode entries (config-only; no code changes — same dynamic pattern as Xiaomi v1.3.0 / Z.AI v1.2.0 / DeepSeek).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 21 new assertions for opencode.

Honest notes

- The alias uses router transport (ccr) because Zen's free models use OpenAI-compatible format (/v1/chat/completions), not

Anthropic native format. This adds a ccr dependency that native-transport aliases (deepseek, xiaomi) don't have.

- Big Pickle is a stealth model — the actual model served may vary (observed as deepseek-v4-flash). This is by design per OpenCode's documentation.
- The same pre-existing `~/api_keys.sh set -u` issue affects the in-process verifier for all providers; authoritative proof is the direct HTTP round trip.
- The 2 pre-existing, environmental `opencode-skill-discovery` failures in `run-proof.sh` remain unchanged (unrelated to this work).

v1.3.0 — 2026-06-19 — Xiaomi MiMo provider alias

Added

- **xiaomi provider alias** — Xiaomi MiMo via the **Anthropic-native endpoint** `https://api.xiaomimimo.com/anthropic` (POST `/anthropic/v1/messages`). Unlike most providers in this toolkit, MiMo exposes a genuine native Anthropic endpoint that accepts `Authorization: Bearer`, so the alias uses **native transport** with no `claude-code-router (ccr)` dependency — the same direct-launch model as deepseek.
- **Model overrides**: `strong = mimo-v2.5-pro` (flagship, 1M context, reasoning, tool-call), `fast = mimo-v2-flash` (256K, cheapest tier). Pinning is deliberate — `models.dev` lists a `mimo-v2.5-pro-ultraspeed` id the **live API does not serve**, so the override guarantees only live-served ids are used.
- **key-aliases.json mapping**: `XIAOMI_MIMO_API_KEY` → `xiaomi` (the user's key-var name does not match the `models.dev` provider's documented `XIAOMI_API_KEY` env).
- **overrides.json pin**: `native transport`, `/anthropic base_url`, `mimo-v2.5-pro / mimo-v2-flash`.
- **Sandbox test coverage**: resolver tests (key-alias mapping, override forces native transport, `/anthropic base_url` beats `catalog /v1`, model pinning beats the stale `ultraspeed` entry, `stale-id-never-selected` guard) + sync e2e tests (env file, alias, `config-dir` + plugins symlink, account-detection exclusion, idempotency, no-secret-leak). Providers test 60 → 69 assertions.
- **Live endpoint verification**: GET `/v1/models` HTTP 200 (10 models); native `/anthropic/v1/messages` round trip HTTP 200 with correct text for both `mimo-v2.5-pro` and `mimo-v2-flash`; tool calling proven (`finish_reason: tool_calls`
 - `reasoning_content`); streaming confirmed. Evidence in `scripts/tests/proof/60-xiaomi-live.txt` (secret-free).

- **Docs:** dedicated xiaomi section in docs/
Provider_Aliases_User_Guide.md (model table, setup, usage, live-verified notes).

Changed

- scripts/providers/key-aliases.json and scripts/providers/overrides.json extended with the xiaomi entries (config-only; no code changes — same dynamic pattern as Z.AI v1.2.0 / DeepSeek).

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Providers test includes 9 new assertions for xiaomi. Live provider verifier 5/5 PASS.

Honest notes

- The only failures in the repo's run-proof.sh are 2 **pre-existing, environmental** opencode-skill-discovery checks, unrelated to Xiaomi (zero opencode files changed by this release; they fail identically when run standalone).
- The in-process LLMsVerifier step reports (unverified) for every provider because ~/api_keys.sh has a pre-existing unrelated unbound variable on a different provider's key under set -u; authoritative proof is the direct native-endpoint round trip (HTTP 200), recorded in the evidence file.

v1.2.0 — 2026-06-19 — Z.AI Coding Plan provider alias

Added

- **zai-coding-plan provider alias** — OpenAI-compatible router transport via https://api.z.ai/api/coding/paas/v4 (Coding Max-Yearly Plan endpoint).
- **Model overrides:** strong = glm-5.2 (flagship 1M context reasoning model, free on plan), fast = glm-4.7 (204k context, tool_call, 0 cost).
- **key-aliases.json mapping:** ZAI_API_KEY → zai-coding-plan (targets the coding plan API endpoint instead of the general z.ai paas endpoint).
- **overrides.json pin:** overrides auto-selected strong/fast models for the coding plan.

- **Sandbox test coverage:** resolver tests (env-key matching, coding endpoint, router transport, glm-5.2/glm-4.7 model selection) + sync e2e tests (env file, alias, model overrides).
- **Live endpoint verification:** HTTP 200 at /models (8 models discovered), curl test of glm-4.7 chat completion confirmed operational.
- **ccr integration:** provider auto-registered in ~/.claude-code-router/config.json as the active default route.

Changed

- overrides.json extended with zai-coding-plan section for model pinning.

Full test suite

- 8/8 test files passed (ALL GREEN), 0 failures. Provider tests include 5 new assertions for zai-coding-plan.

v1.1.0 — 2026-06-16 — Distributed infrastructure + provider verification

Headline: stand up the full LLMsVerifier System on a remote host for heavy testing against **real production LLM services**, plus end-to-end provider aliases proven on two hosts and two transports.

Added

- **containers + challenges submodules** (submodules/) — the distributed-boot orchestrator and its sibling. helix-deps.yaml confirms containers has zero own-org submodule deps.
- **Remote host registration** — config/containers/nezha.env registers nezha.local as a remote boot/test host (SSH key, podman runtime).
- **LLMsVerifier deployment overlays** (config/containers/llmsverifier/):
 - docker-compose.app.yml — the llm-verifier API (cgo image, config mount, /api/health healthcheck, loopback, fail-fast secrets).
 - docker-compose.infra.yml — observability tier: prometheus + grafana (auto-provisioned datasource + dashboard) + node-exporter. **No DBs** (the app uses SQLite; postgres/redis were unused and removed).
 - Dockerfile.nezha / Dockerfile.mv — cgo nested-module builds for the server + the model-verification tool.
 - patches/0001..0005 — upstream LLMsVerifier fixes (see PR #2 below).

- **Deployment guide** config/containers/llmsverifier/README.md and the **Provider Aliases User Guide** docs/Provider_Aliases_User_Guide.md (HTML/PDF/DOCX exports included).
- **QA evidence** docs/qa/20260616-infra/ — verification proofs, endpoint coverage, security posture, observability, per-provider sweeps, dual-host end-to-end alias proofs.

Changed

- **Provider session accent color: orange → purple** across spec, guide, and the long-form doc. (Claude Code 2.1.178 cannot persist a default /color, so this is the documented default + a manual /color purple — a platform limit.)
- `claude-add-account` consolidated onto the shared `cma_link_shared_items` helper (single `CMA_SHARED_ITEMS` source).
- `claude-export-docs` now also emits **DOCX** (HTML/PDF/DOCX).

Fixed (LLMsVerifier — shipped as PR #2, applied to deployed builds)

- **Auth header missing** — verification requests sent no Authorization header → HTTP 401 for every provider. Now Bearer <key>.
- **cohere 405** — switched to the OpenAI-compatible endpoint (`api.cohere.ai/compatibility/v1`). Verifies at score 1.00.
- **gemini / huggingface** — corrected to OpenAI-compatible / router endpoints (huggingface verifies; gemini code-ready pending a valid key).
- **model-id strictness** — verifies a requested id directly when not in the discovered list (no premature `model_not_found`).
- **no /metrics** — added GET `/api/metrics` + `/metrics` (stdlib Prometheus).
- **provider-session sync-state noise** — `cma_run_provider` no longer runs cross-account sync-state on isolated provider dirs.

Verified live (real “Do you see my code?” against production APIs)

- **9 providers verified:** DeepSeek, Groq, Mistral, Cerebras, Novita, NVIDIA, Cohere, Codestral, HuggingFace.
- **Both transports, both hosts:** native (DeepSeek) + router (Novita via ccr) on macOS and on nezha.
- Account-side failures (402/401/429/403) and non-OpenAI providers documented honestly; excluded under “valid users only” but kept fully supported.

Safety

- Provider dirs (~/.claude-prov-*) excluded from account detection — existing claudeN accounts and claude-add-account untouched.
- Secrets only in the keys file + on-host mode-600 .env; never in the repo. All published ports bound to loopback.

v1.0.0 — 2026-06-16 — Dynamic provider-alias generator

First tagged release. claude-providers creates per-provider Claude Code aliases (DeepSeek, Groq, GLM, ...) from your keys file pointed at each provider's strongest model — fully dynamic via models.dev + the LLMsVerifier submodule, hybrid native/claude-code-router transport, full lifecycle + tests + docs. See docs/Provider_Aliases_User_Guide.md.

v1.6.7 — 2026-06-21 — Poe proxy fix for all aliases

Fixed

- **Poe proxy not starting for poe2/poe3** — proxy detection used exact provider ID (poe2_proxy.py) which doesn't exist. Fixed to check base name too (poe_proxy.py for poe2, poe3 aliases).
- **lib.sh**: base proxy detection with `${CMA_PROVIDER_ID%[0-9]*}`
- **alias file**: same fix applied

Verified

- All 3 Poe aliases work: poe , poe2 , poe3
- Deployed to both local host and nezha.local

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 7/8 (pandoc missing — pre-existing)

v1.6.8 — 2026-06-21 — Poe proxy gzip fix

Fixed

- **Poe proxy gzip decompression** — Poe API returns gzip-compressed responses but the proxy tried to read them as UTF-8 without decompressing, causing UnicodeDecodeError. Added gzip decompression for both success and error responses.

Verified

- poe (claude-sonnet-4.6): YES
- poe2 (gpt-5.5): YES
- poe3 (grok-4): Different error (Grok-4 schema validation, not tools format)

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.6.9 — 2026-06-21 — Poe proxy \$ref fix for Grok-4

Fixed

- ****Poe proxy \$ref resolution**** — Claude Code sends tool schemas with ``$ref`references to`$defs``. Grok-4 and some providers don't support ``$ref`in tool schemas`. Added `resolve_refs()` function that extracts ``$defs``, resolves all ``$ref`references` to inline definitions, and removes ``$defs``.

Verified

- poe (claude-sonnet-4.6): YES
- poe2 (gpt-5.5): YES
- poe3 (grok-4): YES (was failing, now works)

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.7.0 — 2026-06-22 — Poe proxy complete fix (all aliases verified)

Fixed

- **Poe proxy shared directory** — the proxy at ~/.local/share/.../proxy/poe_proxy.py was the OLD version without gzip and \$ref fixes. install.sh copies from scripts/ but the shared dir still had the old version. Fixed by ensuring updated proxy is copied to shared directory.
- **install.sh** now copies proxy scripts during installation (already in place)

Verified (all three aliases through full Claude Code flow)

- poe (claude-sonnet-4.6): YES
- poe2 (gpt-5.5): YES
- poe3 (grok-4): YES

Root Cause Analysis

The proxy had three issues: 1. **gzip** — Poe returns gzip-compressed responses, proxy didn't decompress 2. **\$ref** — Claude Code sends tool schemas with \$ref, Grok-4 doesn't support them 3. **shared dir** — Updated proxy wasn't copied to shared directory

All three fixed and verified.

Tests

- Local: 8/8 ALL GREEN
- nezha.local: 8/8 ALL GREEN

v1.7.1 — 2026-06-22 — Full validation + release

Fixed

- **Port-ready check** for proxy startup — replaced sleep 1 with polling loop (lsof -i) ensuring proxy is listening before ccr config is written
- **Claude alias regression test** — 11 assertions proving claudeN aliases use cma_run (no proxy/transformer code), providers use cma_run_provider

- **Command injection fix** in `verify_aliases_live.sh` — replaced `bash -c` subshell with safe indirect expansion

Tests

- Local: **9/9 ALL GREEN** (new: `test_claude.sh` — 11 assertions)
- `nezha.local`: 8/9 (export fails — pandoc missing)

Release

- `v1.7.1` — pushed to github, gitlab, gitflic, gitverse

v1.7.2 — 2026-06-22 — Claude alias verification, full release

Added

- **Claude alias verification** in `verify_aliases_live.sh` — tests `claude1/2/3` alongside provider aliases
- **TOON tested** on all aliases — verified working

🟢

Tests

- Local: **9/9 ALL GREEN**
- `nezha.local`: 8/9 (pandoc missing)
- All `claude1/2/3`: OK
- All provider aliases: verified